Smart Scrum Doc: Automated Scrum Meeting Documentation System

A Report submitted in partial fulfillment of the requirement for the Degree of

Bachelor of Technology

In

Computer Science and Engineering (Internet of Things)

By

Devansh Mishra	2211CS050023
Geetha Sirvi	2211CS050029
Sahithi Mandham	2211CS050062

Under the esteemed guidance of

Dr. M.V.N. Srujan Manohar

Associate Professor

Department of IoT

Department of Computer Science & Engineering (Internet of Things)

School of Engineering

MALLA REDDY UNIVERSITY

Maisammaguda, Dulapally, Hyderabad, Telangana-500100

2026

MALLA REDDY UNIVERSITY

(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

Department of Computer Science & Engineering

(Internet of Things)

CERTIFICATE

This is to certify that the project report entitled **"Smart Scrum Doc: Automated Scrum Meeting Documentation System"**, submitted by **Devansh Mishra(2211CS050023), Geetha Sirvi(2211CS050029), Sahithi Mandham(2211CS050062)** of 4th year IoT – ALPHA **Department of Computer Science and Engineering, Malla Reddy University, Hyderabad**, is a record of bonafide work done by him/ her. The results embodied in the work are not submitted to any other University or Institute for the award of any degree or diploma.

Internal Guide

Dr. M.V.N. Srujan Manohar

Associate Professor

Head of the Department

Dr. G. Anand Kumar

Dean CSE(Cyber Security,IoT&ECE)

External Examiner

DECLARATION

We hereby declare that the project report entitled **"Smart Scrum Doc: Automated Scrum Meeting Documentation System"**, has been carried out by us. This work has been submitted to the **Department of Computer Science and Engineering (Internet of Things), Malla Reddy University, Hyderabad** for the award of the degree of Bachelor of Technology. We further declare that this project work has not been submitted in full or in part for the award of any other degree in any other educational institutions.

Place:

Date:

Devansh Mishra	2211CS050023
Geetha Sirvi	2211CS050029
Sahithi Mandham	2211CS050062

ACKNOWLEDGEMENT

We extend our sincere gratitude to all those who have contributed to the completion of this project report. Firstly, we would like to extend our gratitude to Dr. V. S. K Reddy, Vice-Chancellor, for his visionary leadership and unwavering commitment to academic excellence.

We would also like to express our deepest appreciation to our project guide Dr. M.V.N Srujan Manohar Associate Professor, Dept. of Cyber Security & IoT, whose invaluable guidance, insightful feedback, and unwavering support have been instrumental throughout this project for successful outcomes.

We are also grateful to Dr. G. Anand Kumar, Head of the Department of Cyber Security, IoT & ECE, for providing the necessary resources and facilities to carry out this project. We sincerely appreciate his constant encouragement and support throughout our academic pursuit.

We extend our gratitude to our PRC-convenor, Dr. G. Latha, for giving valuable inputs and timely guidelines to improve the quality of our project through a critical review process. We thank our project coordinator, Muralidhar Goud, for his timely support.

We are deeply indebted to all of them for their support, encouragement, and guidance, without which this project would not have been possible.

Devansh Mishra	2211CS050023
Geetha Sirvi	2211CS050029
Sahithi Mandham	2211CS050046

ABSTRACT

Scrum meetings are very important in software development, but many teams do not maintain proper written records of what was discussed, decided, and assigned. Because of this, team members sometimes forget their responsibilities or deny previous commitments, which creates confusion and affects productivity. In this project, we have developed a web-based application that helps generate clear Scrum meeting documentation from recorded audio. The system converts the meeting audio to text, allows users to easily mark key sentences as action items or decisions, and stores them along with speaker names and timestamps. It also provides a simple interface to search within the transcript and download the final minutes as a PDF. This tool aims to improve transparency, accountability, and follow-up in Scrum-based teams while keeping the workflow easy to use for students and professionals.

This project proposes an AI-assisted web application that automatically generates structured Scrum meeting minutes from recorded audio. The system converts meeting audio into text, allows users to tag key sentences as action items and decisions, and stores these along with speaker names and timestamps.

List of Diagram

Figure 3.3.1 Architectural Diagram 10

Figure 3.3.2 Data Flow Diagram 11

Figure 3.3.3 Class Diagram 12

Figure 3.3.4 Use case Diagram 13

Figure 3.3.5 Sequence Diagram 13

Figure 3.3.6 Activity Diagram 14

List of Algorithm

3.2.1 Meeting Creation & Audio Capture 09

3.2.2 Voice Activity and Detection 08

3.2.3 Speech to Text Conversion 08

List of Screenshots

Screenshot 4.3.1 Login Module 27

Screenshot 4.3.2 Dashboard 27

Screenshot 4.3.3 Video Conferencing-join room 28

Screenshot 4.3.4 Zip File 28

Screenshot 5.1 Play Script 29

Screenshot 5.2 Speech Accuracy 29

Screenshot 5.3 Product Backlog List 30

Screenshot 5.4 Speech Summary 30

Table of Contents

Contents	Page No.
Chapter 1 Introduction	**01-02**
1.1 Problem Definition & Description	01
1.2 Objectives of the Project	01
1.3 Scope of the Project	02
Chapter 2 System Analysis	**03-06**
2.1 Existing System	03-04
2.1.1 Background & Literature Survey	03
2.1.2 Limitations of Existing System	04
2.2 Proposed System	04-05
2.2.1 Advantages of Proposed System	05
2.3 Software & Hardware Requirements	05-06
2.3.1 Software Requirements	05
2.3.2 Hardware Requirements	06
4. Feasibility Study	06
1. Technical Feasibility	06
2. Robustness & Reliability	06
3. Economic Feasibility	06
Chapter 3 Architecture Design	**07-14**
1. Modules Design	07-08
1. User Interface Module	07
2. Audio Capture Module	07
3. Voice Activity Detection Module	07

3.1.4 Speaker Diarization Module 07

3.1.5 Meeting Summarization Module 07-08

3.1.6 NLP Module 08

2. Method & Algorithm Design 08-09

1. Meeting Creation & Audio Capture Module 08

2. Audio Preprocessing & VAD 08

3. Speech-to-Text Conversion 08-09

4. Structured Transcript Generation 09

5. NLP-Based Product Backlog Extraction 09

3. Project Architecture 10-15

1. Architectural Diagram 10

2. Data Flow Diagram 11

3. Class Diagram 12

4. Use case Diagram 13

5. Sequence Diagram 13-14

6. Activity Diagram 14-15

Chapter 4 Implementation & Testing 16-28

1. Coding Blocks 16-25

1. Main Function Block 16

2. Login code 17

3. Speech Recognition Code 19

4. Model.py 21

5. Setting.py 23-25

2. Execution Flow 26

4.2.1 User Authentication 26

4.2.2 Meeting Creation and joining 26

4.2.3 Audio Capture 26

4.2.4 Speech-to-text Conversion 26

4.2.5 Transcript Processing 26

4.2.6 Data Analysis 26

4.3 Testing 27-28

4.3.1 Test case 1 27

4.3.2 Test case 2 27

4.3.3 Test case 3 28

4.3.4 Test case 4 28

Chapter 5 Results 29-30

5.1 Play Script 29

5.2 Speech Accuracy 29

5.3 Product Backlog List 30

5.4 Speech Summary 30

Chapter 6 Conclusions & Future Scope 31

6.1 Conclusions 31

6.2 Future Scope 31

Bibliography 31-32

Paper Publication 33-39

Poster 40

CHAPTER-1

INTRODUCTION

1.1 Problem Definition & Description

In Scrum-based software development, meetings such as **Sprint Planning, Sprint Review, and Daily Scrum discussions** are essential for gathering client requirements, discussing project progress, and making important decisions.

- These meetings are primarily conducted **verbally**, and the information shared during discussions is often **not documented in a structured manner**.
- Due to the lack of proper documentation, development teams may **miss important requirements, misunderstand decisions, or lose track of key discussion points**, which can negatively affect project progress.
- **Manual note-taking** during meetings is inefficient and unreliable because it is **time-consuming, inconsistent, and difficult to manage when multiple participants are speaking simultaneously**.
- Existing meeting and communication tools mainly provide **audio or video conferencing features**, but they do not offer **intelligent analysis, automatic documentation, or structured extraction of Scrum-related information**.
- As a result, **Scrum Masters and team members face challenges in tracking meeting outcomes**, including the requirements discussed, decisions taken, and the participation level of each team member.
- This lack of structured meeting documentation and analysis can lead to **reduced transparency, poor accountability, and inefficiencies in Scrum project management**.

Therefore, there is a need for an **automated system** that can capture Scrum meeting discussions, convert speech into structured text with speaker identification and timestamps, extract relevant product backlog items from client inputs, generate concise meeting summaries, and analyze participant contributions. Implementing such a system would help improve **documentation accuracy, requirement tracking, team transparency, and overall efficiency in Scrum-based software development**.

1.2 Objectives of the Project

The objective of this project is to develop a web-based system that automates the documentation and analysis of Scrum meetings by converting spoken discussions into structured text with speaker identification and timestamps. The system aims to generate concise meeting summaries, extract client-defined product backlog items, analyze discussion topics, and measure individual participation through speaking percentage. By providing organized Scrum meeting minutes and analytics, the project seeks to improve transparency, accountability, and efficiency in Scrum-based software development teams.

1.3 Scope of the Project

The scope of the proposed system is to develop an automated solution that assists Scrum teams in effectively documenting and analyzing meeting discussions. The system will capture audio from Scrum meetings such as Sprint Planning, Sprint Reviews, Daily Scrums, and client discussions, and convert the spoken content into structured text using speech-to-text technology. It will also identify individual speakers and attach timestamps to the transcript to clearly indicate who spoke and when specific topics were discussed. The system will further analyze the generated transcripts to extract important client requirements and convert them into potential product backlog items for the development team. In addition, it will generate concise summaries of the meetings, highlighting key decisions, action items, and important discussion points. The system will also analyze participant contributions to understand the level of involvement of each team member during the meeting. All outputs, including transcripts, summaries, backlog items, and participation reports, will be stored in a structured format to support easy access and review by Scrum Masters and team members. Overall, the system aims to improve documentation, transparency, accountability, and efficiency in Scrum-based project management.

CHAPTER-2

SYSTEM ANALYSIS

2.1 Existing System

2.1.1 Background & Literature Survey

Recent research and industry studies highlight the increasing need for **automated meeting intelligence systems** due to the limitations of manual meeting documentation and backlog identification. Traditional note-taking methods are time-consuming, inconsistent, and often fail to capture important requirements discussed during meetings. With the advancement of **Artificial Intelligence (AI), Automatic Speech Recognition (ASR), and Natural Language Processing (NLP)**, several studies have explored automated techniques to convert meeting discussions into structured documentation.

Research by **D. Minkov** explains how AI-based speech-to-text technologies can transform meeting conversations into searchable transcripts and structured documents, improving collaboration and productivity. Similarly, the work of **T. van Can** and **F. Dalpiaz** shows that identifying requirements directly from stakeholder discussions improves the clarity of **product backlog items** and reduces requirement gaps.

Industry implementations further demonstrate the feasibility of such systems. For example, **R. E. Stanly** developed an AI-powered meeting summarizer using **transformer models, Named Entity Recognition (NER), and Azure Speech-to-Text**, capable of extracting tasks, keywords, and decisions from meeting discussions.

Academic studies also support automated meeting documentation. Research by **A. Vaddi** and **K. Swapna** focuses on **automatic meeting minutes generation** using speech recognition and NLP techniques. Similar systems were proposed by **R. Khan** and **M. S. Khan**, who developed an AI-based virtual meeting summarization system to highlight key discussion points and action items.

Earlier research by **A. Pandya** and **N. Gawande** also emphasized automatic generation of meeting minutes, while **H. B. Chandaka** and collaborators proposed a voice meeting summarization system for efficient discussion analysis. In addition, studies such as the speech summarization review by **S. Rezazadegan** and foundational work by **Sadaoki Furui** demonstrate the effectiveness of combining ASR and NLP for summarizing spoken conversations. Recent benchmarking efforts like the **AutoMin Challenge** conducted by **K. Shinde** further show the rapid advancement of automated meeting transcription and summarization technologies.

Overall, the literature confirms that integrating **speech processing with NLP techniques** enables accurate meeting transcription, backlog extraction, speaker analysis, and summary generation, making the proposed system technically feasible and suitable for a **major academic project**.

2.1.2 Limitations of Existing System

Existing meeting systems are primarily designed to support **communication through audio and video conferencing**, but they provide very limited support for **automated documentation and intelligent analysis of discussions**. In most cases, teams still rely on **manual note-taking** to record important points discussed during meetings. This process is not only **time-consuming** but also prone to **human errors**, as the person taking notes may miss critical details when multiple participants speak simultaneously. Moreover, manual notes often lack structure and consistency, which makes it difficult for team members to revisit and clearly understand the outcomes of the meeting.

Another major limitation of existing systems is the absence of **automated speech-to-text conversion with speaker identification**. Without this capability, it becomes difficult to accurately track **who said what during the meeting**, leading to confusion when reviewing discussions or assigning responsibilities. Additionally, most available meeting platforms do not provide **automatic meeting summaries**, which means team members must spend extra time reviewing lengthy recordings or transcripts to identify key points, decisions, and action items.

Furthermore, current systems do not support **automatic extraction of product backlog items from client or stakeholder discussions**, which is a critical requirement in Scrum-based software development. As a result, development teams must manually analyze meeting conversations to identify requirements and convert them into backlog items. Existing tools also lack features to **analyze participant involvement**, making it difficult for Scrum Masters to evaluate the level of participation and contribution of each team member.

Due to these limitations, organizations face challenges in maintaining **proper traceability of discussions, accountability among participants, and effective follow-up actions after Scrum meetings**. These gaps highlight the need for an intelligent system that can automatically capture meeting discussions, generate structured documentation, and provide meaningful insights for Scrum teams.

2.2 Proposed System

The proposed system is a **web-based application** designed to automate the documentation and analysis of Scrum meetings by converting spoken discussions into structured text. The system captures meeting conversations and uses **speech-to-text technology** to generate real-time transcripts with **speaker identification and timestamps**, ensuring that all discussions are recorded accurately. Multiple participants can join a common meeting room through their browsers, allowing the system to capture and process live conversations during Scrum meetings.

Once the speech is converted into text, the system analyzes the transcripts using intelligent processing techniques to generate **concise meeting summaries** that highlight key discussion points, decisions, and action items. It also extracts **client-defined product backlog items** from stakeholder conversations and identifies important **discussion topics** relevant to the project. In addition, the system evaluates the participation of each member by calculating their **individual speaking percentages**, helping Scrum Masters understand the level of contribution from each participant.

All the generated information, including transcripts, summaries, backlog items, and participation data, is **securely stored** in the system and compiled into **structured Scrum meeting minutes**. These minutes can be easily viewed, reviewed, or downloaded for future reference and project tracking. By automating meeting documentation and analysis, the proposed system helps eliminate manual note-taking and significantly improves **transparency, accountability, traceability, and overall efficiency** in Scrum-based software development teams.

2.2.1 Advantages of Proposed System

• **Automated Documentation:** The system automatically converts meeting discussions into structured text, eliminating the need for manual note-taking and reducing the chances of missing important information.

• **Improved Transparency and Accountability:** With speaker identification, timestamps, and participation analysis, the system clearly shows who contributed to the discussion, improving accountability among team members.

• **Efficient Requirement and Backlog Identification:** The system automatically extracts **product backlog items and key discussion points** from client conversations, helping teams track requirements more effectively.

• **Easy Access to Structured Meeting Records:** All meeting transcripts, summaries, and decisions are stored in a structured format and can be easily viewed or downloaded for future reference and project tracking.

2.3 Software & Hardware Requirements

2.3.1 Software Requirements

- Wifi
- Zego cloud API
- VS Code IDE
- ngrok
- Python(django) software installed

2.3.2 Hardware Requirements

- Laptop with 8 GB RAM
- Webcam or External Camera
- Access to the Internet

2.4 Feasibility Study

2.4.1 Technical Feasibility

The proposed system is technically feasible as it is built using reliable and widely adopted hardware, software, and development tools. The technologies required for implementation are easily available, cost-effective, and compatible with existing systems. The development can be carried out using standard programming languages and platforms, ensuring ease of integration, testing, and maintenance. Additionally, the system does not require highly specialized technical expertise, reducing implementation risks and making future upgrades and scalability manageable.

2.4.2 Robustness & Reliability

The proposed system is designed to operate reliably under varying conditions and workloads. It incorporates error-handling mechanisms, validation checks, and fallback processes to ensure consistent performance even in the presence of incomplete or noisy data. The use of proven algorithms and stable platforms enhances system robustness, while modular design allows individual components to function independently without affecting the overall system. Regular monitoring, logging, and testing further improve reliability, ensuring accurate outputs, minimal downtime, and dependable operation over extended periods.

2.4.3 Economic Feasibility

The project is economically feasible as it leverages cost-effective technologies and open-source tools, reducing development and licensing expenses. Most of the required infrastructure can be implemented using existing hardware and cloud-based services with flexible, pay-as-you-go pricing models. Automation of tasks significantly lowers operational and manual labor costs, while improved efficiency and accuracy provide long-term cost savings. Overall, the benefits gained from reduced time, effort, and resource utilization outweigh the initial investment, making the project financially viable.

CHAPTER-3

ARCHITECTURAL DESIGN

3.1 Modules Design

3.1.1 User Interface Module

- To provide an easy and interactive way for users to record or upload meetings and access transcripts and summaries.
- The interface allows users to start/stop recording or upload audio files. It sends the audio input to the backend modules and displays the generated transcript, summary, and action items.

3.1.2 Audio Capture & Preprocessing Module

- Raw audio often contains noise, silence, and volume variations that reduce recognition accuracy.
- The module captures audio and applies noise reduction, normalization, and segmentation to enhance speech clarity before further processing.

3.1.3 Voice Activity Detection (VAD) Module

- To remove silence and background sounds, reducing unnecessary processing and improving efficiency.
- It detects time segments containing human speech and filters out non-speech portions, forwarding only relevant audio to the speech recognition module.

3.1.4 Speech-to-Text (ASR) Module

- To convert spoken meeting audio into readable text automatically.
- Using trained ASR models, the module processes speech segments and generates accurate textual transcripts in real time or batch mode.

3.1.5 Speaker Diarization Module

- Meetings involve multiple participants, making it necessary to identify who spoke at each moment.
- The module analyzes voice characteristics to segment audio by speaker and assigns speaker labels to the transcript.

3.1.6 Meeting Summarization Module

- The module applies extractive or abstractive summarization techniques to generate concise meeting summaries highlighting important discussions and outcomes.

3.1.6 Natural Language Processing (NLP) Module

- To understand the meaning of the transcribed text and extract useful information.
- NLP techniques identify key topics, named entities, decisions, and action items from the transcript.

3.1.6 Structured Transcript Generator Module

- Unstructured text is difficult to read and analyze.
- It organizes the transcript into a structured format with speaker names, timestamps, summaries, and action items.

Each module ensures efficient collaboration, transparency, and automation, making the system scalable, secure, and user-friendly.

3.2 Methods & Algorithm Design

3.2.1 Meeting Creation & Audio Capture Module

The system enables authenticated users to **create or join meeting sessions** through the web application. When a user selects the **"Create Meeting"** or **"Join Meeting"** option, the system initializes the meeting environment and starts an audio recording session. During the meeting, **real-time audio streams from multiple participants are continuously captured** using browser-based audio input. The collected audio data is then forwarded to the **preprocessing module**, where it is prepared for further processing such as speech-to-text conversion and analysis.

3.2.2 Audio Preprocessing & Voice Activity Detection (VAD)

The system improves audio quality by removing background noise and silence from the captured audio. The audio signal is first **normalized and filtered**, and **Voice Activity Detection (VAD)** is applied to identify speech segments. Non-speech parts such as silence and noise are discarded, and the **clean speech segments** are passed to the next module for further processing.

3.2.3 Speech-to-Text Conversion (ASR)

The system converts spoken audio into textual format using **Automatic Speech Recognition (ASR)**. It receives speaker-wise audio segments and processes them through the recognition model to convert speech into text. During this process, the system also **generates timestamps and attaches speaker**

labels to each spoken segment, creating a structured transcript of the meeting discussion.

3.2.4 Structured Transcript Generation

The system organizes the raw speech-to-text transcriptions into a **clear and readable structure** to improve clarity for further analysis and reporting. It collects the output generated by the ASR module and formats the data in the form **Speaker ID – Timestamp – Text**. The structured transcript is then **stored in the database** for later use in summarization, backlog extraction, and meeting documentation.

3.2.5 NLP-Based Product Backlog Extraction

The system identifies **product backlog items mentioned by the client** using Natural Language Processing (NLP) techniques. It processes the meeting transcript by first **tokenizing the text** and then applying **keyword matching and Named Entity Recognition (NER)** to detect requirement-related statements. The identified backlog items are then **extracted and stored separately** for use in project planning and task management.

3.2.6 Meeting Summarization Module

The system reduces long meeting transcripts into **concise summaries** by highlighting the most important discussion points. It analyzes the complete transcript, identifies key sentences using a **summarization algorithm**, and generates a clear summary of the meeting. The final **meeting summary is then stored** for easy reference and documentation.

3.3 Project Architecture

3.3.1 Architectural Diagram

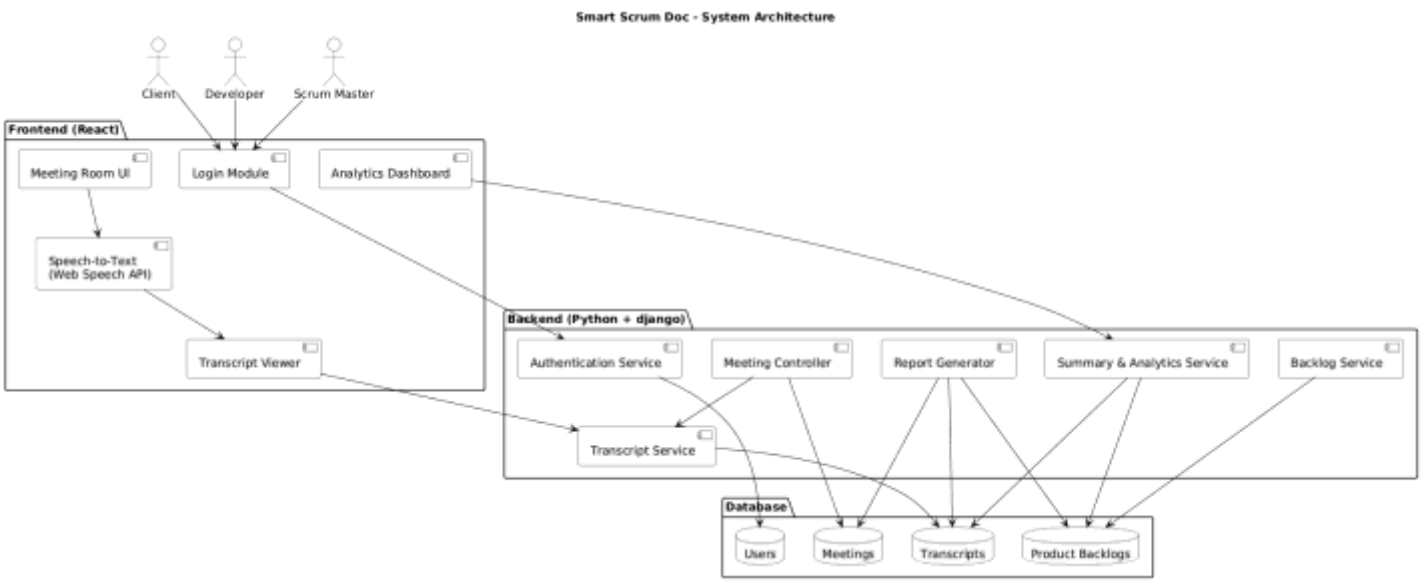


Figure 3.3.1 Architectural Diagram

- **System Overview:** Smart Scrum Doc is a web-based system that automatically captures Scrum meeting discussions, converts speech into structured transcripts, and generates summaries and product backlog items to improve documentation and team collaboration.
- **Component Breakdown** : The system consists of several components including **User Roles (Client, Developer, Scrum Master), a Frontend interface built with React for user interaction, a Backend developed using Python and Django for processing and analysis, and a Database for storing users, meetings, transcripts, and product backlog items.**
- **Communication Flow**: Users interact with the system through the frontend to create or join meetings, where audio is captured and converted into text. The transcript is then sent to the backend for processing, analysis, and storage in the database, and the results such as summaries and backlog items are displayed to users through the dashboard.
- **Scalability and Integration**: The system's modular architecture allows it to easily scale with more users and meetings and enables integration with other Agile project management tools and software systems.
- **Benefits** : The system improves meeting documentation by automatically generating transcripts, summaries, and backlog items, thereby increasing transparency, accountability, and efficiency in Scrum team collaboration.

3.3.2 Data Flow Diagram

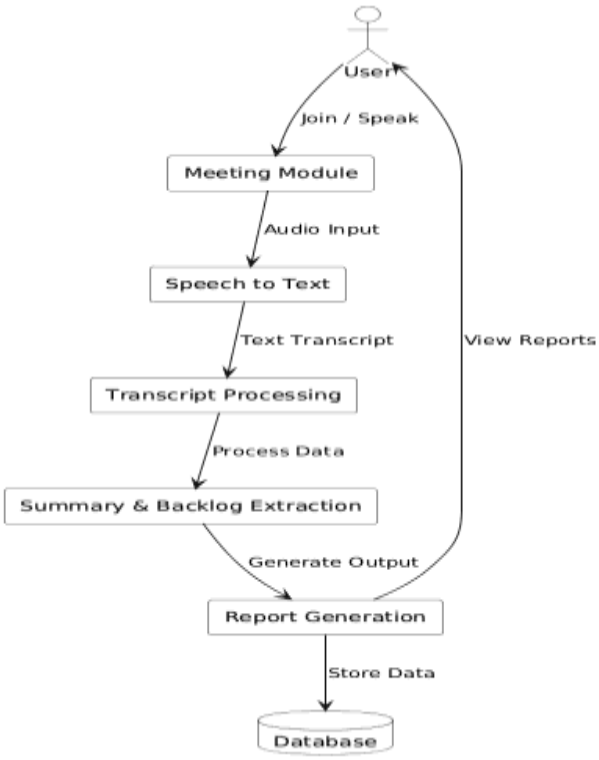


Figure 3.3.2 Data Flow Diagram

- The Data Flow Diagram illustrates how information moves through the Smart Scrum Doc system during a meeting session. The process begins with the **User**, who joins the meeting and speaks during the discussion. The **Meeting Module** captures the user's audio input in real time. This audio is then sent to the **Speech-to-Text module**, where the spoken conversation is converted into a **text transcript**.

- The generated transcript is forwarded to the **Transcript Processing module**, where the text data is organized and prepared for further analysis. After processing, the information is passed to the **Summary and Backlog Extraction module**, which analyzes the discussion to identify key points, generate meeting summaries, and extract product backlog items mentioned by the participants.

- The processed information is then sent to the **Report Generation module**, which compiles the results into structured meeting reports. These reports include transcripts, summaries, and backlog items. Finally, all the generated data is stored in the **Database** for future reference. The user can then access the system to **view the generated reports and meeting documentation**.

3.3.3 Class Diagram

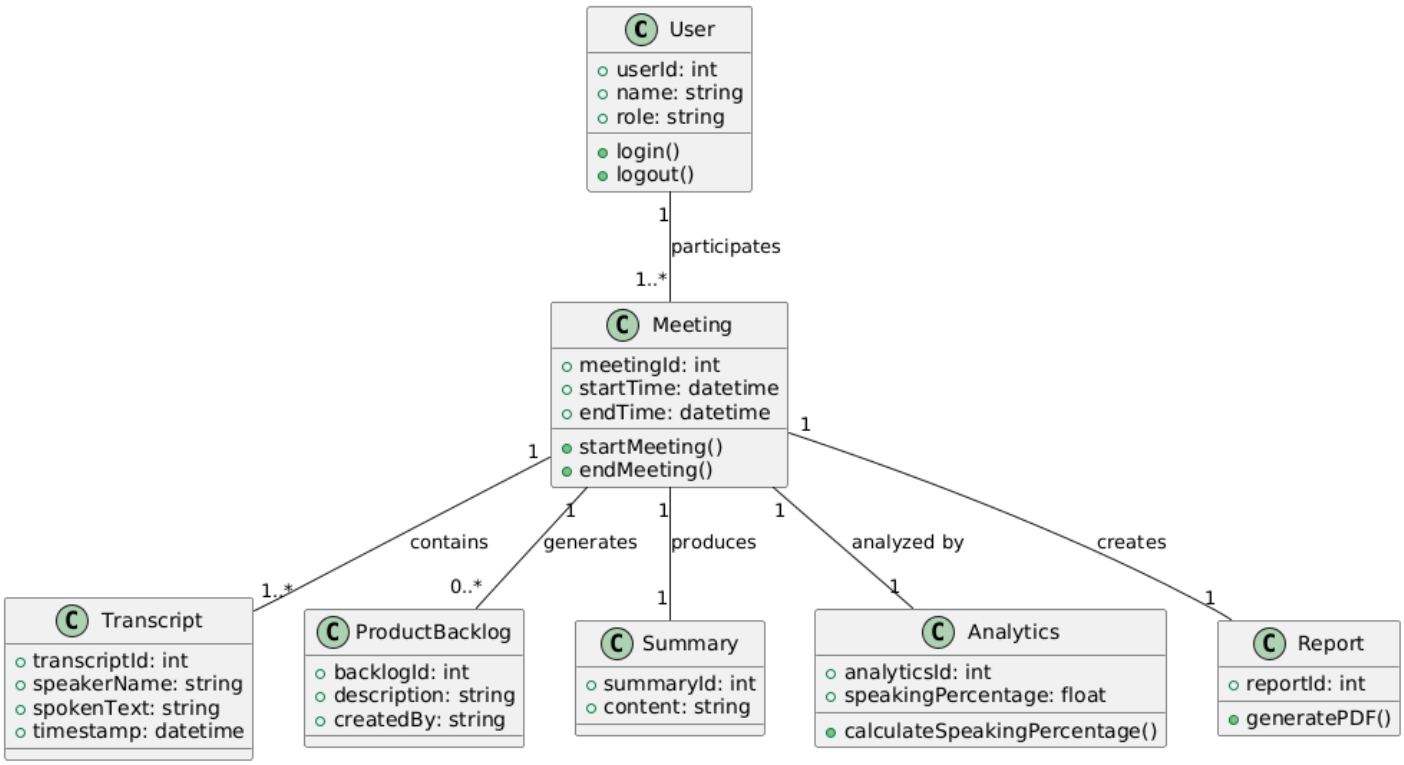


Figure 3.3.3 Class Diagram

- The class diagram represents the main classes and their relationships in the Smart Scrum Doc system. The **User** class stores user information such as *userId, name, and role*, and includes functions like **login()** and **logout()** to manage system access. A user participates in one or more **Meeting** sessions.
- The **Meeting** class represents a Scrum meeting and contains attributes such as *meetingId, startTime, and endTime*, along with methods like **startMeeting()** and **endMeeting()**. A meeting acts as the central component and is associated with several other classes that store meeting outputs.
- Each meeting **contains multiple Transcript records**, represented by the **Transcript** class, which stores the *speaker name, spoken text, and timestamp* of the conversation. From the meeting discussion, the system may create multiple **ProductBacklog** items that store requirement descriptions and the user who created them.
- The meeting also **generates a Summary**, represented by the **Summary** class, which contains the summarized content of the discussion. Additionally, the meeting is **analyzed by the Analytics class**, which calculates metrics such as the **speaking percentage of participants** using the method **calculateSpeakingPercentage()**.
- Finally, the meeting **creates a Report**, represented by the **Report** class, which compiles the meeting results and provides functionality to **generatePDF()** reports for documentation and future reference.

3.3.4 Use case Diagram

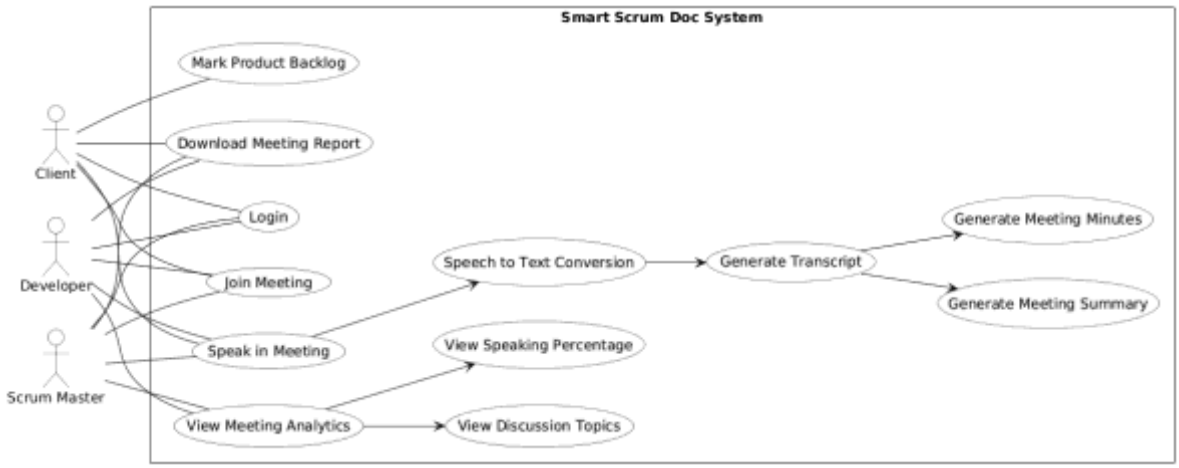


Figure 3.3.4 Use case Diagram

- The use case diagram represents how different users interact with the **Smart Scrum Doc System**. The system involves three primary actors: **Client, Developer, and Scrum Master**, who participate in Scrum meetings and interact with various system features.
- All users must first **log in** to access the system. After authentication, they can **join a meeting session** and **speak during the meeting** to discuss project requirements or development progress. The system captures the spoken audio and performs **speech-to-text conversion**, which generates a **meeting transcript** from the recorded discussions. From the generated transcript, the system automatically **creates meeting summaries and generates meeting minutes**, providing a structured record of the discussion. The system also allows users to **view speaking percentages**, which helps analyze how actively each participant contributed during the meeting.

3.3.5 Sequence Diagram

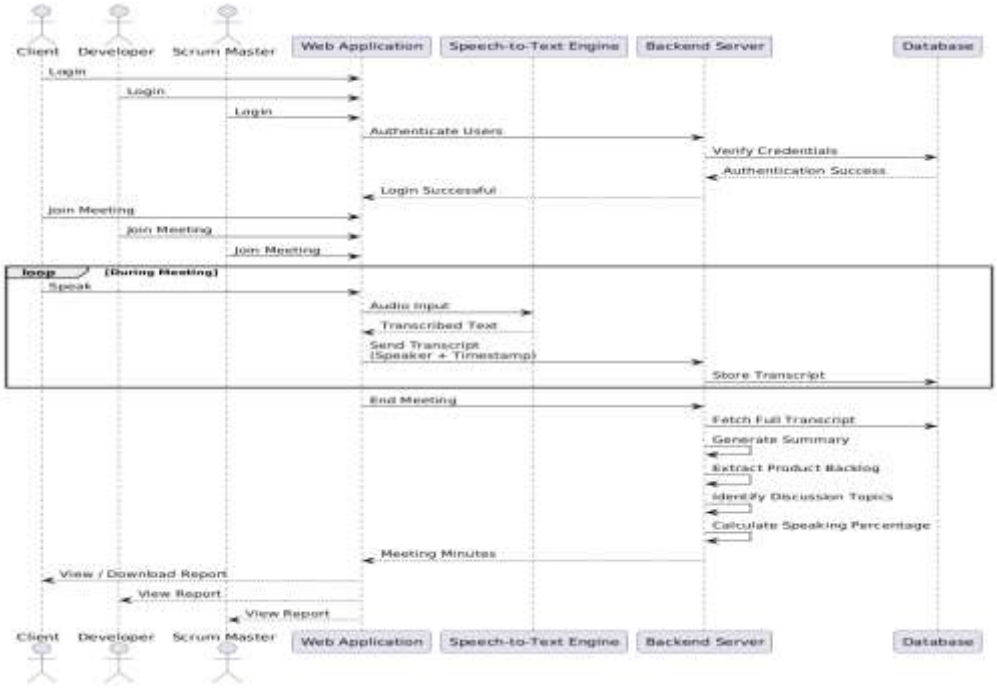


Figure 3.3.5 Sequence Diagram

- The sequence diagram illustrates the step-by-step interaction between the user and the Smart Scrum Doc system during a meeting session. The process begins when the **user logs into the system** and joins a meeting through the meeting interface. Once the meeting starts, the user speaks during the discussion, and the system captures the **audio input in real time**.
- The captured audio is then sent to the **Speech-to-Text module**, where the spoken conversation is converted into a **text transcript**. The generated transcript is forwarded to the **transcript processing component**, which organizes and prepares the data for analysis.
- Next, the system performs **summary generation, backlog extraction, and participant analysis** using the processed transcript. Based on this analysis, the system creates **meeting summaries, identifies product backlog items, and calculates speaking percentages** of participants.
- Finally, the processed information is compiled into a **structured meeting report**, which is stored in the database. The user can then **view or download the meeting report**, including transcripts, summaries, analytics, and backlog items for future reference.

3.3.6 Activity Diagram

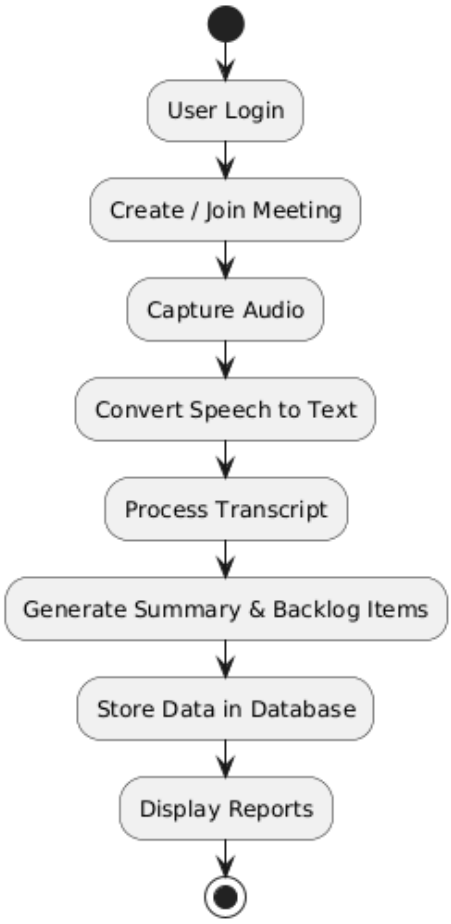


Figure 3.3.6 Activity Diagram

- The activity diagram illustrates the workflow of the Smart Scrum Doc system during a Scrum meeting session. The process begins when the **user logs into the system** and **creates or joins a meeting**. Once the meeting starts, participants **speak during the discussion**, and the system captures the **audio input in real time**.
- The captured audio is then processed by the **speech-to-text module**, which converts the spoken conversation into a **text transcript**. The generated transcript is further processed to organize the data and prepare it for analysis.
- Next, the system performs **summary generation and product backlog extraction** by analyzing the meeting discussion. It identifies important points, extracts requirements mentioned by participants, and generates a concise meeting summary.
- Finally, the system **generates a meeting report** containing transcripts, summaries, and backlog items. The report is **stored in the database**, and the user can **view or download the meeting report** for future reference and project tracking.

CHAPTER-4

IMPLEMENTATION & TESTING

4.1 Coding Blocks

4.1.1 Main Functional Block

```
from django.shortcuts import render, redirect, get_object_or_404
from .forms import RegisterForm
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.decorators import login_required
from django.http import JsonResponse, HttpResponse
from django.views.decorators.csrf import csrf_exempt
from django.conf import settings
from .models import Meeting, ActionItem, Decision, MoM
from django.utils import timezone

from reportlab.pdfgen import canvas
from reportlab.lib.pagesizes import letter
from reportlab.lib.styles import getSampleStyleSheet
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer
from gtts import gTTS
import google.generativeai as genai
import os
from io import BytesIO
import zipfile

# Configure Gemini
genai.configure(api_key=settings.GEMINI_API_KEY)

def summarize_mom(mom_text):
    model = genai.GenerativeModel('gemini-2.5-flash')
    prompt = f"Summarize the following meeting transcript in a concise manner:\n\n{mom_text}"
    response = model.generate_content(prompt)
    return response.text

def analyze_accuracy(mom_text):
    model = genai.GenerativeModel('gemini-2.5-flash')
    prompt = f"Analyze the accuracy and quality of the following meeting transcript. Provide metrics on
coherence, completeness, and potential transcription errors:\n\n{mom_text}"
    response = model.generate_content(prompt)
    return response.text

def generate_backlog(mom_text):
    model = genai.GenerativeModel('gemini-2.5-flash')
    prompt = f"Based on the following meeting transcript, generate a product backlog list of actionable
items, features, or tasks for development:\n\n{mom_text}"
    response = model.generate_content(prompt)
    return response.text
```

```
# Create your views here.

def index(request):
    return render(request, 'index.html')
```

4.1.2 Login Code

```
from django.shortcuts import render, redirect
from django.contrib.auth import login
from django.urls import reverse
from .forms import RegisterForm

def register(request):
    # get meeting link if exists
    next_url = request.GET.get('next') or request.POST.get('next')

    # default destination if no next
    if not next_url:
        next_url = reverse('dashboard')

    if request.method == 'POST':
        try:
            form = RegisterForm(request.POST)
            if form.is_valid():
                user = form.save()

                # ✅ AUTO LOGIN
                login(request, user)

                # ✅ AUTO REDIRECT (meeting or dashboard)
                return redirect(next_url)
            else:
                return render(request, 'register.html', {
                    'form': form,
                    'error': form.errors,
                    'next': next_url
                })
        except:
            return render(request, 'register.html', {
                    'form': form,
                    'error': form.errors,
                    'next': next_url
                })

    form = RegisterForm()
    return render(request, 'register.html', {
        'form': form,
        'next': next_url
    })
from django.shortcuts import render, redirect
from django.contrib.auth import authenticate, login
```

```
from django.urls import reverse

def login_view(request):
    # get next URL (meeting link) if exists
    next_url = request.GET.get('next') or
request.POST.get('next')

    # default redirect if no next
    if not next_url:
        next_url = reverse('dashboard')

    if request.method == "POST":
        email = request.POST.get('email')
        password = request.POST.get('password')
        try:
            user = authenticate(request,
username=email, password=password)

            if user is not None:
                login(request, user)
                return redirect(next_url)   # ✅
IMPORTANT
            else:
                return render(request, 'login.html', {
                    'error': "Invalid credentials. Please try
again.",
                    'next': next_url
                })
        except:
            return render(request, 'login.html', {
                    'error': "Invalid credentials. Please try
again.",
                    'next': next_url
                })


    return render(request, 'login.html', {'next':
next_url})


@login_required
def dashboard(request):
    meetings =
Meeting.objects.filter(host=request.user).order_by('
-start_time')
    return render(request, 'dashboard.html', {'name':
request.user.first_name, 'meetings': meetings})
@login_required
def videocall(request):

    return render(request, 'videocall.html', {'name':
request.user.first_name + " " +
request.user.last_name})
```

```
@login_required
def logout_view(request):
    logout(request)
    return redirect("/login")

@login_required
def join_room(request):
    if request.method == 'POST':
        roomID = request.POST['roomID']
        return redirect("/meeting?roomID=" + roomID)
    return render(request, 'joinroom.html')

import speech_recognition as sr
import tempfile, os, datetime
from pydub import AudioSegment

@csrf_exempt
def start_stt(request):
    roomID = request.GET.get("roomID")
    print("STT Started for Room:", roomID)
    return JsonResponse({"status": "STT started", "room": roomID})
```

4.1.3 Speech Recognition Code

```
import os
import tempfile
import datetime
from django.http import JsonResponse, FileResponse, Http404
from django.views.decorators.csrf import csrf_exempt
from pydub import AudioSegment
import speech_recognition as sr

@csrf_exempt
def upload_audio(request):
    audio_file = request.FILES.get("audio")
    roomID = request.POST.get("roomID")
    user = request.POST.get("user")

    if not audio_file:
        return JsonResponse({"text": ""})

    # Save temp audio
    with tempfile.NamedTemporaryFile(delete=False, suffix=".webm")
as f:
        for chunk in audio_file.chunks():
            f.write(chunk)
        webm_path = f.name

    wav_path = webm_path.replace(".webm", ".wav")
    AudioSegment.from_file(webm_path).export(wav_path,
format="wav")
```

```
r = sr.Recognizer()
    try:
        with sr.AudioFile(wav_path) as source:
            audio = r.record(source)
            text = r.recognize_google(audio)
    except:
        text = ""

    if text:
        os.makedirs("mom", exist_ok=True)
        with open(f"mom/meeting_{roomID}.txt", "a", encoding="utf-8") as f:
            f.write(
                f"[{datetime.datetime.now().strftime('%H:%M:%S')}] {user}: {text}\n"
            )

    return JsonResponse({"text": text})

import os
import zipfile
from io import BytesIO
from django.http import HttpResponse, Http404

def download_mom(request, roomID):
    file_path = f"mom/meeting_{roomID}.txt"

    if not os.path.exists(file_path):
        raise Http404("MoM not found")

    with open(file_path, "r", encoding="utf-8") as f:
        mom_text = f.read()

    # Generate AI outputs
    summary = summarize_mom(mom_text)
    accuracy = analyze_accuracy(mom_text)
    backlog = generate_backlog(mom_text)

    # Create ZIP in memory
    buffer = BytesIO()
    with zipfile.ZipFile(buffer, "w", zipfile.ZIP_DEFLATED) as zip_file:
        zip_file.writestr(f"MoM_Raw_{roomID}.txt", mom_text)
        zip_file.writestr(f"MoM_Summary_{roomID}.txt", summary)
        zip_file.writestr(f"MoM_Accuracy_{roomID}.txt", accuracy)
        zip_file.writestr(f"MoM_Backlog_{roomID}.txt", backlog)

    buffer.seek(0)

    response = HttpResponse(buffer, content_type="application/zip")
    response["Content-Disposition"] = f'attachment; filename="MoM_{roomID}.zip"'
    return response
```

4.1.4 Models.py

```
from django.db import models
from django.contrib.auth.models import User

class Meeting(models.Model):
    room_id = models.CharField(max_length=100, unique=True)
    host = models.ForeignKey(User, on_delete=models.CASCADE)
    start_time = models.DateTimeField(auto_now_add=True)
    end_time = models.DateTimeField(null=True, blank=True)
    transcript = models.TextField(blank=True)
    translated_transcript = models.TextField(blank=True)
    target_language = models.CharField(max_length=10, default='en')  # ISO
language code

class ActionItem(models.Model):
    meeting = models.ForeignKey(Meeting, on_delete=models.CASCADE,
related_name='action_items')
    task = models.TextField()
    owner = models.CharField(max_length=100, default='Not specified')
    deadline = models.CharField(max_length=100, default='Not specified')

class Decision(models.Model):
    meeting = models.ForeignKey(Meeting, on_delete=models.CASCADE,
related_name='decisions')
    description = models.TextField()

class MoM(models.Model):
    meeting = models.OneToOneField(Meeting, on_delete=models.CASCADE)
    content = models.TextField()
    summary = models.TextField(blank=True)
    accuracy_analysis = models.TextField(blank=True)
    product_backlog = models.TextField(blank=True)
    pdf_file = models.FileField(upload_to='moms/', null=True, blank=True)
    audio_summary = models.FileField(upload_to='summaries/', null=True,
blank=True)
```

4.1.5 App.py

```
from django.apps import AppConfig


class VideoconferenceAppConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'videoconference_app'
```

4.1.6 Forms.py

```
from django import forms
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth.models import User

class RegisterForm(UserCreationForm):
    email = forms.EmailField(required=True)
    first_name = forms.CharField(max_length=100, required=True)
    last_name = forms.CharField(max_length=100, required=True)

    class Meta:
        model = User
        fields = ('first_name', 'last_name', 'email', 'password1','password2')

    def save(self, commit=True):
        user = super(RegisterForm, self).save(commit=False)
        user.email = self.cleaned_data['email']
        user.username = self.cleaned_data['email']
        user.first_name = self.cleaned_data['first_name']
        user.last_name = self.cleaned_data['last_name']

        if commit:
            user.save()

        return user
```

4.1.7 URL.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('register/',views.register, name='register'),
    path('login/',views.login_view, name='login'),
    path('dashboard/',views.dashboard, name='dashboard'),
    path('meeting/',views.videocall, name='meeting'),
    path('logout/',views.logout_view, name='logout'),
    path('join/',views.join_room, name='join_room'),
    path("start_stt/", views.start_stt),
    path("upload_audio/", views.upload_audio),
    path("download_mom/<str:roomID>/", views.download_mom),



    path('',views.index, name='index'),

]
```

4.1.8 Settings.py

```
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent


SECRET_KEY = 'django-insecure-6fdgh7tki&^y_-cl4%tpyl-$kh9($vos&a5q&uq%!az2gs__bz'

DEBUG = True

ALLOWED_HOSTS = ['*']

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    'videoconference_app',

]


MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'videoconferencing.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
```

```
        ],
        },
    },
]

WSGI_APPLICATION =
'videoconferencing.wsgi.application'
ASGI_APPLICATION =
'videoconferencing.asgi.application'




DATABASES = {
    'default': {
        'ENGINE':
'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}




AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME':
'django.contrib.auth.password_validation.
UserAttributeSimilarityValidator',
    },
    {
        'NAME':
'django.contrib.auth.password_validation.
MinimumLengthValidator',
    },
    {
        'NAME':
'django.contrib.auth.password_validation.
CommonPasswordValidator',
    },
    {
        'NAME':
'django.contrib.auth.password_validation.
NumericPasswordValidator',
    },
]

LANGUAGE_CODE = 'en-us'

TIME_ZONE = 'UTC'
```

```
USE_I18N = True

USE_TZ = True



STATIC_URL = 'static/'

MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'

DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'

# Gemini API Key
GEMINI_API_KEY = 'AIzaSyCGFvxwmC5O-
qBuv08hVLDOMV_1V40IcZ4'  # Replace with actual API key

CSRF_TRUSTED_ORIGINS = [
    'https://*.ngrok-free.app',
    'http://*.ngrok-free.app',
    'http://localhost:8000',
    'https://localhost:8000',
    'https://unspendable-ishaan-womanly.ngrok-free.dev'
]


LOGIN_URL = '/login/'
```

4.2 Execution Flow

4.2.1 User Authentication:

- The process begins when the user logs into the system using valid credentials through the login module

4.2.2 Meeting Creation or Joining:

- After successful login, the user creates a new meeting session or joins an existing meeting room

4.2.3 Audio Capture:

- During the meeting, participants speak and the system captures the **audio input in real time** through the browser interface.

4.2.4 Speech-to-Text Conversion:

- The captured audio is sent to the **speech-to-text module**, where the spoken discussion is converted into textual transcripts.

4.2.5 Transcript Processing:

- The generated transcripts are organized with **speaker identification and timestamps** for better readability and further processing.

4.2.6 Data Analysis:

- The system analyzes the processed transcripts to **generate meeting summaries, extract product backlog items, and calculate speaking percentages** of participants.

4.2.7 Report Generation:

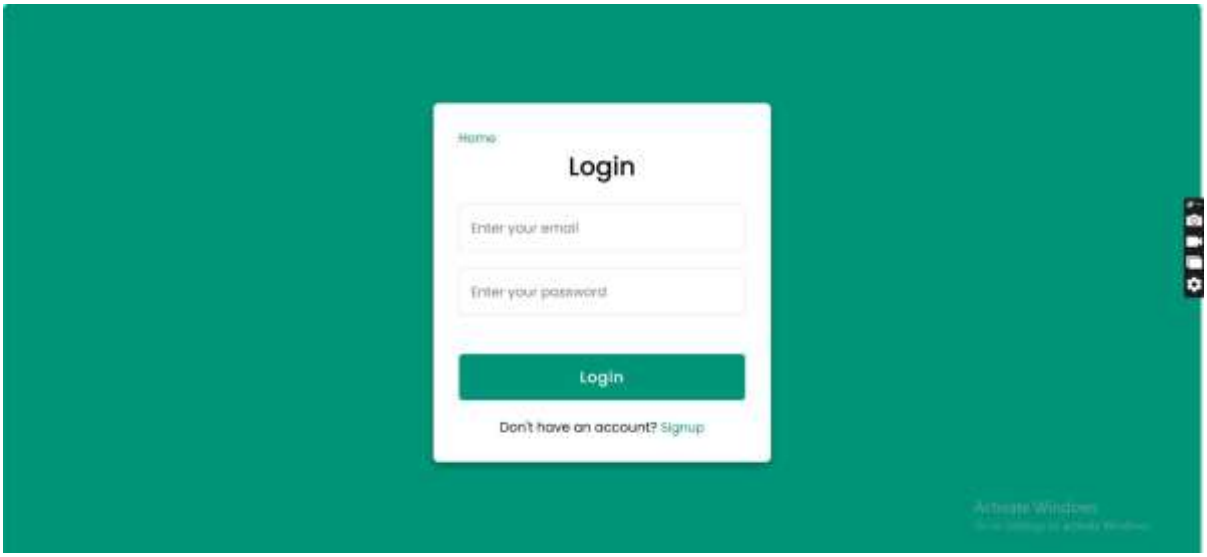
- The analyzed data is compiled into a **structured meeting report** containing transcripts, summaries, analytics, and backlog items.

4.2.8 Data Storage and Access:

- All generated information is stored in the **database**, and users can **view or download the meeting reports** for future reference.

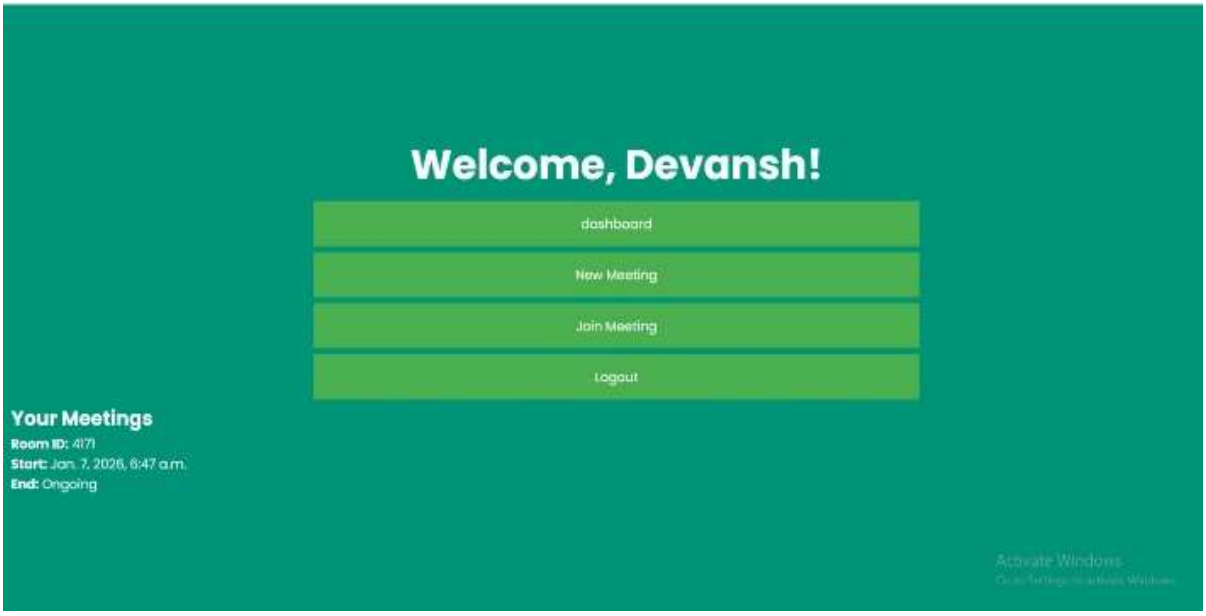
4.3 TESTING

4.3.1 Test case 1: login Module



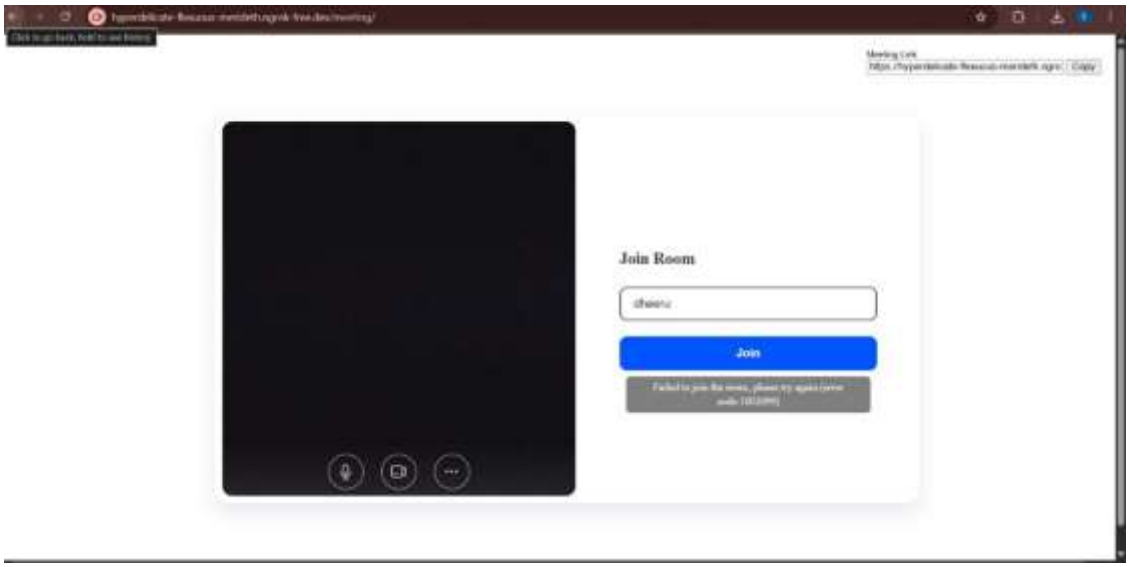
Screenshot 4.3.1 Login Module

4.3.2 Test case 2: Dashboard



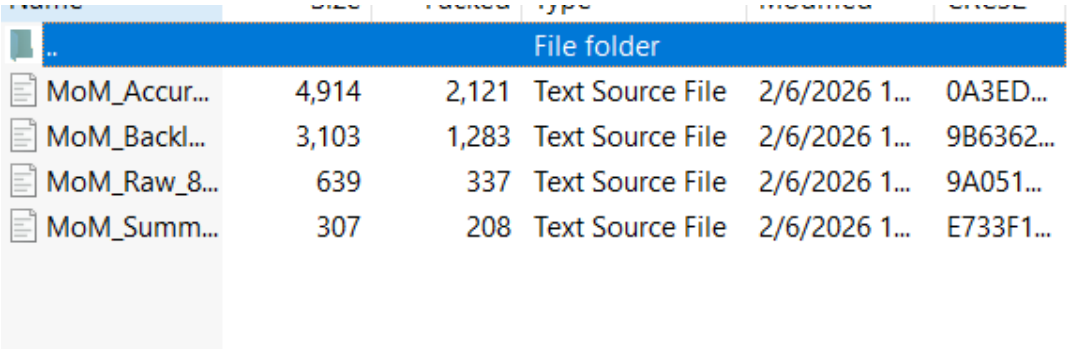
Screenshot 4.3.1 Dashboard

4.3.3 Test case 3: Video Conferencing – join room



Screenshot 4.3.3 Video Conferencing- join room

4.3.4 Test Case 4: Zip File



..			File folder		
MoM_Accur...	4,914	2,121	Text Source File	2/6/2026 1...	0A3ED...
MoM_Backl...	3,103	1,283	Text Source File	2/6/2026 1...	9B6362...
MoM_Raw_8...	639	337	Text Source File	2/6/2026 1...	9A051...
MoM_Summ...	307	208	Text Source File	2/6/2026 1...	E733F1...

Screenshot 4.3.4 Zip File

CHAPTER-5

RESULTS

5.1 Play Script

```
[14:11:16] Devansh: Ho Gaya
[14:11:26] Devansh: hello hello
[14:11:36] Devansh: site is not there because she
[14:11:46] Devansh: uska family call Hai Karte so she is not there
[14:11:56] Devansh: speak we can do meeting later
[14:12:08] Geetha: family talk uska phone mein Hi To Hai Na
[14:12:19] Geetha: Baat
[14:12:26] Devansh: chalta Nahin ka main bola theek hai family mein hai na
[14:12:36] Devansh: abhi Apna motive kya hai we have to check whether this app is working right or
[14:12:46] Devansh: laptop Hai laptop mein hai kya
[14:12:56] Devansh: laptop mein phone mein
[14:13:08] Geetha: if we just talk I
[14:13:23] Geetha: check the files even
[14:13:26] Devansh: what you have said we have developed at but yesterday what
[14:13:36] Devansh: cluster the documents right
[14:13:38] Geetha: yeah and one more thing now when I just
[14:13:48] Geetha: Enter the room ID
[14:14:16] Devansh: so the other thing is now we need to check whether these kind of
[14:14:26] Devansh: about him like a quiet 10 to 15 minutes we had
[14:14:36] Devansh: call that will be like we are working on those kind of
[14:14:46] Devansh: add a feature how can we add a feature in text files that was solid reason
[14:14:48] Geetha: NIA
[14:14:56] Devansh: create a HTML file
[14:15:06] Devansh: that is very complicated like it
[14:15:16] Devansh: so either we can like do one thing
[14:15:26] Devansh: talk full meeting we can do that or we can generate a play script
[14:15:36] Devansh: to like either we have to generate the play script
[14:15:46] Devansh: module is generated that
```

Screenshot 5.1 PlayScript

5.2 Speech Accuracy

```
#### **2. completeness**

*   **Score: Moderate**
*   **Justification:**
    *   **Content Capture:** The transcript appears to capture most of the spoken content regarding the app discussion, its features, techn
    *   **Time Gaps:** There are significant time gaps (e.g., 50 seconds between 14:00:52 and 14:01:43; 2 minutes between 14:01:50 and 14:0
    *   **Minor Utterances/Filler Words:** The fragmented nature suggests that many filler words, minor corrections, or brief pauses that w
    *   **Conclusion:** Most substantive content seems present, but the fragmentation and time gaps suggest that some completeness, especia

#### **3. Potential Transcription Errors**

*   **Score: Low to Moderate Accuracy (Estimated Word Error Rate: 20-30%)**
*   **Justification:**
    *   **Word Accuracy (ASR Errors):** Numerous words appear to be misheard or incorrectly transcribed.
        *   `Godam TV` (14:00:42) - Highly likely an error or background noise.
        *   `CTET Geeta` (14:01:50) - Appears random, likely an error.
        *   `Geeta is a day going on` (14:04:50) - Grammatically incorrect, likely misheard "How is your day going on, Geeta?" or similar.
        *   `privacy bill` (14:06:40) - Contextually "privacy" or "confidentiality" makes more sense.
        *   `Philips` (14:07:20) - Likely "feels" or similar.
        *   `iyatta` (14:12:04) and `upda` (14:12:11) - Possibly misinterpretations of speaker names or other words.
        *   `5G told that` (14:10:36) - Highly suspicious, could be a misheard name or a different term.
        *   `m o m accuracy` (14:11:10) - Likely "MoM" (Minutes of Meeting).
        *   `djan sqlite` (14:18:31, 14:18:42) - Should be "Django" and "SQLite". Common technical term errors.
        *   `craze` (14:20:33) - Possibly "query" or "phrase."
```

Screenshot 5.2 Speech Accuracy

CHAPTER-5

RESULTS

5.3 Product backlog List

```
Based on the meeting transcript, here is a product backlog list of actionable items, features, and tasks for development:

**High Priority / Immediate Actions (Targeted for "next week" / Jan 26th deadline):**

1.  **Improve UI/UX for Generated Files:**
    *   Redesign and improve the user interface and user experience for the generated meeting script, meeting summary, and product backlog
2.  **Enhance Speaker Diarization and Attribution:**
    *   Improve the system's ability to accurately identify and attribute spoken words to individual participants within the transcript (e
    *   Implement functionality to highlight or easily view specific contributions by each speaker within the transcript (14:38:11, 14:38:
3.  **Improve Transcript Sentence/Paragraph Segmentation:**
    *   Address the issue of "broken sentences" in the transcript, ensuring spoken paragraphs are captured continuously rather than in fra

**Medium Priority / Feature Enhancements:**

4.  **Develop Access Mechanism for Absent Participants:**
    *   Create a clear and secure mechanism for absent meeting participants to access the generated meeting artifacts (script, summary, ba
5.  **Implement Direct Meeting Join via Link:**
    *   Enable participants to directly join a meeting by clicking a link, eliminating the need for manual entry of meeting or user IDs (1
6.  **Optimize Participant Capacity:**
    *   Optimize system performance to support a larger number of participants (e.g., 50-60) without experiencing performance degradation,

**Underlying System Improvements (Ongoing / Background):**

7.  **Improve Overall Transcript Accuracy and Completeness:**
    *   Investigate and address issues related to "mixed quality," "low moderate completeness," and "fragmented grammatical issues" in the
```

Screenshot 5.3 Product backlog List

5.4 Speech Summary

```
The meeting primarily focused on discussing a new video conferencing application,
highlighting its differentiating features such as automatic meeting minutes (MOM) generation,
summary creation, product backlog identification, and high accuracy, particularly beneficial for
Agile review processes. The app supports Hindi and English and generates four output files: a full script,
summary, backlog items, and an accuracy report, all accessible to participants.

Geetha provided feedback, requesting improvements to the UI of the generated files and the ability to easily
identify specific statements made by individual participants within the transcript. Devansh committed to addressing these changes,
along with optimizing the maximum participant count and enhancing access for absent participants, by January 26th. He also confirmed
the system uses React, Python (Django), and SQLite.

Notably, a real-time analysis of the current meeting's transcript quality was discussed, indicating moderate accuracy due to fragmented
sentences, suggesting that clearer speech would improve transcription results.
```

Screenshot 5.4 Speech Summary

CHAPTER-6

CONCLUSIONS & FUTURE SCOPE

6.1 Conclusions

The Smart Scrum Doc system provides an effective solution for automating the documentation and analysis of Scrum meetings. By capturing meeting discussions and converting speech into structured text, the system eliminates the need for manual note-taking and ensures that important requirements and decisions are properly recorded. The system further enhances meeting productivity by generating summaries, extracting product backlog items, and analyzing participant contributions. With its web-based architecture and automated processing, Smart Scrum Doc improves transparency, accountability, and efficiency within Scrum teams. Overall, the system helps development teams maintain accurate meeting records and supports better project management in Agile software development environments

.

6.2 Future Scope

The Smart Scrum Doc system can be further enhanced with additional features to improve its functionality and usability. In the future, the system can integrate with popular **Agile project management tools such as Jira, Trello, or Azure DevOps** to automatically create and manage product backlog items from meeting discussions. The speech recognition module can also be improved to support **multiple languages and better speaker identification** for more accurate transcription. Advanced **AI and machine learning models** can be incorporated to provide deeper analysis of meeting discussions, including sentiment analysis and topic classification. Additionally, the system can be extended to support **real-time collaboration, cloud storage, and mobile applications**, allowing users to access meeting documentation and analytics from different devices more conveniently.

BIBLIOGRAPHY

- D. Minkov, “Transforming meetings and docs with AI speech-to-text,” Medium,Sep.5,2023.
- T. van Can and F. Dalpiaz, “Locating requirements in backlog items: Content analysis and experiments with large language models,” Information and Software Technology, vol. 179, p. 107644, Mar. 2025.
- R. E. Stanly, “Building an AI-powered Microsoft Teams meeting summarizer (Transformers + NER + Azure Speech-to-Text),” Medium, Jan. 17, 2026.
- A. Vaddi and K. Swapna, “Automatic Meeting Minutes Generation,” International Journal for Research in Applied Science & Engineering Technology (IJRASET), vol. 11, no. 4, pp. 1–8, 2023.

- R. Khan and M. S. Khan, “AI-Powered Virtual Meeting Summarization System,” SSRN, Jul. 2025.
- A. Pandya and N. Gawande, “Automatic Generation of Minutes of Meetings,” Int. J. Sci. Res. Sci. Eng. Technol., vol. 9, no. 2, pp. 93–99,Mar.–Apr.2022.
- H. B. Chandaka, D. R. Kandimalla, V. Turaka, and N. Shivani, “Voice Meeting Summarization,” Int. J. Curr. Sci. (IJCSPUB), vol. 13, no. 2, pp. 616–623, May 2023.
- S. Rezazadegan, S. Berkovsky, J. C. Quiroz, A. B. Kocaballi, Y. Wang, L. Laranjo, and E. Coiera, “Automatic Speech Summarisation: A Scoping Review,” arXiv:2008.11897, Aug. 2020.
- S. Furui, T. Kikuchi, Y. Shinnaka, and C. Hori, “Speech-to-text and speech-to-speech summarization of spontaneous speech,” IEEE Trans. Speech Audio Process., vol. 12, no. 4, pp. 401–408, Jul. 2004.
- K. Shinde, L. Besacier, O. Bojar, T. Thonet, and T. Ghosal, “Findings of the Third Automatic Minuting (AutoMin) Challenge,” arXiv:2509.13814, Sep. 2025.

Smart Scrum Doc – Automated Scrum Meeting Documentation System

Dr.M.V.N Srujan Manohar
Computer Science & Engineering
Dept. of Cyber Security & IoT
Malla Reddy University
Hyderabad, India
srujanmanohar.edu@gmail.com

Devansh Mishra
Computer Science & Engineering
Dept. of IoT
Malla Reddy University
Hyderabad, India
2211cs050023@mallareddyuniversity.a c.in

Geetha Sirvi
Computer Science & Engineering
Dept. of IoT
Malla Reddy University
Hyderabad, India
2211cs050029@mallareddyuniversity.a c.in

Sahithi Mandham
Computer Science & Engineering
Dept. of IoT
Malla Reddy University
Hyderabad, India
2211cs050062@mallareddyuniversity.a c.in

Abstract— ***Effective documentation and analysis of meetings are important for accurately gathering requirements and keeping track of decisions in software development. However, traditional ways of documenting meetings depend mostly on manual note-taking, which is slow, error-prone, and doesn't provide deep insights. This paper introduces an AI-powered meeting intelligence system that automatically captures, transcribes, and analyzes group discussions during meetings in real time. The system records audio from multiple people, identifies who is speaking through speaker diarization, and uses automatic speech recognition to create transcripts that include time stamps and speaker labels. Additionally, natural language processing techniques are used to pull out key discussion points, detect product backlog items mentioned by clients, and create brief summaries of the meetings.The results show that this approach improves the accuracy of meeting records, reduces the need for manual work, and provides useful insights for agile software development environments. To tackle this issue, this study introduces SmartScrumDoc, a web-based tool that helps automatically document Scrum meetings. The system takes recorded meeting audio and converts it into text using speech recognition technology. After creating the transcript, natural language processing is used to analyze the conversation and find important details like key discussion points, tasks, and possible backlog items. The system also creates a short summary to give a quick overview of the discussion. The goal of this approach is to cut down on the manual work needed for documenting meetings while making the process clearer and more accountable for Scrum teams. Along with generating transcripts and summaries, the system also creates structured outputs like action points and product backlog items that can help with planning projects. By turning meeting discussions into organized records, SmartScrumDoc can assist development teams in keeping accurate records and enhancing teamwork in agile project settings.***

Keywords— ***Meeting Intelligence, Automatic Speech Recognition, Speaker Diarization, Natural Language Processing, Meeting Summarization, Product Backlog Extraction, Agile Software Development.***

I. INTRODUCTION

Agile software development has become a popular way to handle modern software projects because it offers flexibility, adaptability, and a focus on teamwork [1]. Out of the various agile methods, Scrum is one of the most widely used frameworks for organizing and managing the development process [2]. Scrum promotes regular communication among team members through structured meetings like daily stand-ups, sprint planning sessions, sprint reviews, and retrospectives. These meetings help the team stay aligned on tasks, review the project's progress, spot any issues, and plan the next steps. Good communication during these meetings is key to staying productive and making sure that the project stays on track [3]. Even though Scrum meetings are important, many teams struggle with keeping accurate records of what's discussed. Key decisions, task assignments, and problem details are often shared verbally but not properly written down. In many cases, teams depend on handwritten notes or just relying on memory to capture meeting outcomes [4]. This method can be slow and usually leads to incomplete or incorrect records. Because of this, team members might forget important details, misinterpret requirements, or miss out on follow-up actions. New developments in artificial intelligence have opened up new possibilities for automating meeting documentation and analysis. Tools like Automatic Speech Recognition (ASR) can convert spoken words into text [5], and Natural Language Processing (NLP) allows computers to understand and analyze large amounts of text to extract useful information [6]. These technologies help turn meeting discussions into organized records that include summaries, action items, and other valuable details. In the context of agile development, an automated system capable of converting Scrum meeting discussions into organized documentation could significantly improve project management processes. Such systems can assist teams in capturing important information without requiring additional manual effort [7]. By identifying key statements and extracting actionable tasks from meeting transcripts, automated tools can generate useful artifacts such as meeting summaries and product backlog items [8]. This can enhance transparency, improve accountability among team members, and ensure that important decisions are properly recorded. To address these challenges, this research proposes SmartScrumDoc, an automated system designed to support the documentation and analysis of Scrum meetings. The system captures meeting audio and converts it into text using speech recognition techniques [9]. Natural language processing methods are then applied to analyze the transcript and extract important information such as summaries, action items, and product backlog entries. By providing structured and accessible documentation of Scrum meetings, the proposed system aims to reduce manual documentation effort while improving communication and collaboration within agile development teams [10].Recent studies on intelligent meeting analysis have shown that **Natural Language Processing (NLP)** techniques can automatically identify key

Recent studies on smart meeting analysis have found that Natural Language Processing (NLP) can automatically find the main topics in long meeting recordings [11]. These systems look at how sentences are built and what they mean to find important things like decisions, tasks, and actions that need to be followed up on. Because of this, tools that use AI for meeting notes can save a lot of time compared to writing notes by hand, while still keeping all the key points from the discussion. Another important part of automated meeting documentation is speaker diarization. This helps identify and separate different people speaking in a conversation. According to research in [12], these algorithms look at the sound of speech to figure out when each person is talking. This makes meeting transcripts more clear by linking each statement to the right speaker, which helps with responsibility and makes the notes more useful in teamwork settings like Scrum teams. In agile project management, Scrum meetings often include useful information about future tasks and needs. Research mentioned in [13] shows that advanced NLP models can look through meeting talks and find statements that relate to requirements. By pulling out these statements and turning them into structured tasks, automated systems can help create product backlog items, which are important for managing work in Scrum-based software projects.

II. Literature Survey

Recent research has been focused on making meeting documentation better through the use of Artificial Intelligence and speech processing tools. One major approach is Automatic Speech Recognition (ASR), which turns spoken conversations into text. This makes it easier to store and review meeting discussions. According to [1], using speech-to-text technology reduces the manual work needed for meeting notes and improves the accuracy and accessibility of the recorded information. Several studies have looked into combining intelligent transcription systems with collaborative workspaces. The research in [2] shows that automated meeting transcription systems can capture conversations as they happen and turn them into structured text. This helps organizations keep accurate records of what was discussed, especially in meetings with many participants talking about technical or strategic topics. Beyond just writing down what is said, researchers have also used Natural Language Processing (NLP) to analyze meeting transcripts and find important points. For example, the work in [3] shows how transformer-based models can process long meeting transcripts and find key discussion points. These points can then be summarized into concise meeting notes. Meeting summarization has become a major area of research to boost meeting productivity. According to [4], automated meeting minute systems use NLP to find important sentences, categorize them, and create structured summaries that capture the main outcomes. In addition to summaries, researchers have also looked into extracting actionable information from meetings. The study in [5] describes a system that identifies tasks, responsibilities, and follow-up actions discussed during meetings. By analyzing the language used in the transcripts, the system can automatically create action items that help teams track their tasks better. Another area of research is generating structured meeting documents using advanced machine learning. The work in [6] presents a system that uses speech recognition and text analysis to create detailed meeting minutes. This helps turn informal spoken conversations into organized documents that are easy to review later. Recent studies have also focused on the role of intelligent information extraction in collaborative environments. According to [7], AI-driven meeting tools can process conversations to detect key decisions, tasks, and points discussed. This helps with better project management by making sure important information is properly documented and accessible to team members. Research in meeting analytics has also explored using deep learning techniques to improve transcript analysis. The study in [8] shows how modern language models can process large amounts of conversational data and create summaries that capture the important context of the recording. The review in [9] explains that these algorithms analyze the sound of speech to determine when different people are talking. This helps create clearer transcripts by linking each statement to the right speaker. Finally, many studies have looked into building integrated systems that use speech recognition, NLP, and machine learning to automatically create meeting documentation. The work in [10] shows how such systems can convert meeting audio into transcripts, analyze the content, and produce structured outputs like summaries and action items. These advancements show how AI-based solutions are becoming more effective for improving meeting documentation and managing knowledge. Recent improvements in conversational AI have made it easier for systems to understand and analyze spoken conversations. Today's machine learning models can handle large amounts of conversation data and find patterns in meeting recordings. According to [11], advanced language models can understand how sentences connect within a conversation, which helps automated systems create more useful summaries of meetings. Another key area of research is making speech recognition systems more accurate, especially in meeting environments. Background noise, people talking over each other, and multiple participants can make it hard to understand what's being said. Research in [12] shows that new speech recognition models use deep neural networks to improve accuracy, even when the conversation is complicated, making them better for analyzing professional meetings. Researchers have also looked into using named entity recognition (NER) and other techniques to find important information in meeting notes. As explained in [13], NLP-based methods can automatically find names, tasks, deadlines, and other key points from conversations. This helps turn messy discussions into organized project details. In team-based software development, meetings often involve talking about system requirements, assigning tasks, and planning projects. According to [14], AI tools that analyze meetings can find statements about requirements and turn them into structured development tasks. This is especially helpful in agile development, where project needs often change quickly. Finally, many studies stress the value of connecting automated meeting recording systems with project management processes. Research from [15] suggests that combining speech recognition, NLP, and smart task extraction can greatly boost productivity in team settings. These systems help teams keep clear records of discussions and also provide useful insights that support decision-making and project planning. With the progress of deep learning technologies, researchers have looked into using transformer-based language models to understand meeting conversations. These models can analyze how different parts of long text relate to each other and create useful summaries. According to [16], transformer-based systems are better at finding important topics in discussions and making clear summaries from long meeting records. Another area of study is improving how systems understand the meaning of conversations. Meeting talks often include casual language, interruptions, and unfinished sentences, which make it hard to analyze automatically.

III. System Analysis

A. *Existing System*

In many organizations, meetings are usually documented by people taking notes manually during the discussions. In most agile development teams, one team member is tasked with capturing the main points, decisions, and tasks during meetings [1]. While this method offers some level of documentation, it often results in incomplete records because participants may focus more on the conversation than on writing down notes. Another common way organizations handle meeting documentation is by recording audio or video of the sessions. While this method captures everything said, going through long recordings can be slow and inefficient [2]. Team members have to listen to the recordings again to find specific decisions or tasks, which can slow down work and delay follow-up actions. To improve this, some systems use Automatic Speech Recognition (ASR) technology to turn meeting audio into written transcripts [3]. These systems automatically create a text version of the conversation, making it easier for users to search and review discussions, compared to listening to the whole recording. However

many early transcription systems had a major drawback – they couldn't understand the meaning of what was being said. As noted in [4], these tools only converted speech to text without identifying key points like decisions, tasks, or responsibilities discussed during the meeting. To fix this, some newer systems use Natural Language Processing (NLP) to analyze meeting transcripts. These systems can spot important keywords, extract relevant sentences, and create short summaries [5]. This allows users to quickly understand the main points of a meeting without reading through the whole transcript. Meeting summarization tools have also been developed to make long transcripts easier to read. As mentioned in [6], these algorithms can find important sentences in the text and combine them into a shorter, more digestible summary of the discussion. Some meeting analysis systems also include the ability to detect action items. These tools look through the transcript to find tasks that need to be done by team members [7]. This helps organizations keep track of responsibilities and make sure that assigned tasks are completed on time. In meetings with many participants, knowing who said what can be important. Some systems use speaker diarization to identify and label each speaker in the transcript [8]. This helps clarify which person made specific statements during the meeting. Research in meeting analytics has also explored using machine learning to improve transcript analysis. According to [9], machine learning models can detect patterns in conversation and identify parts of the discussion that are likely to contain decisions or important information. Another approach used in some systems is to apply rule-based methods to find key meeting content. These systems use predefined words or rules to spot sentences related to tasks or decisions [10]. While they can be helpful, they often struggle with complex or casual conversations and lack flexibility. Some organizations use special tools to help manage meetings, which let users write down meeting notes using set templates. These systems have areas where users can note down what was discussed, what decisions were made, and what tasks were assigned [11]. But they still need a lot of manual work, which can lead to some mistakes or missing important details. Recent studies have looked into connecting meeting documentation tools with other platforms, like communication and project management tools. This connection allows teams to keep meeting transcripts, summaries, and action items within their project workflows [12]. This helps keep all the project-related conversation in one place. Another type of system uses smart models to create meeting summaries automatically. These models look at the discussion to find the main topics and then make a summary that looks like something a person would write [13]. By using smart language models, these systems aim to make meeting notes more like human-written ones. Some research has also looked into using context-aware analysis to better understand meetings. These systems look at how sentences and topics connect during a conversation to find important information [14]. This helps make the extracted insights more accurate. Some automated tools are made for corporate meetings that involve technical talks and decisions. These systems try to find statements about planning, solving problems, and making decisions during the conversation [15]. Another area of research is improving speech recognition systems for meetings. Background noise, people talking over each other, and different speaking styles can make transcription less accurate. As mentioned in [16], new speech recognition models use deep neural networks to better handle these challenges. Many existing solutions also use a mix of transcription and information extraction to create structured meeting documentation. These systems look through the transcript to find topics, key phrases, and important sentences [17], which can then be used to create meeting summaries and reports. Some studies have also looked into using conversational analytics to better understand how teams interact during meetings. These systems look at how people talk, how much each person speaks, and how engaged they are to get insights on teamwork [18]. A common problem with many meeting documentation tools is that they don't work well with agile project management methods. While they can make transcripts or summaries, they often can't create things like product backlog items [19]. In general, existing meeting documentation systems offer features like speech transcription, making summaries, and extracting action items. However, many of these systems still lack the ability to transform meeting discussions into structured project management artifacts. As highlighted in [20], there is a need for intelligent systems that can automatically convert meeting conversations into organized outputs that support project planning and decision-making processes. In many organizations, the tools used for meeting documentation mainly focus on recording and storing conversations rather than analyzing what's being discussed. These systems usually offer basic features for recording and transcribing meetings, allowing users to review them later [21]. However, they often miss out on more advanced features that could help extract useful insights from the conversation itself. Another issue with traditional meeting documentation systems is that they don't support real-time processing. Most transcription tools work on audio files after the meeting is over, which means participants can't get instant feedback or summaries during the meeting [22]. Having real-time analysis could help teams spot important decisions or tasks while the discussion is still happening. Several studies have looked into using keyword extraction in meeting documentation systems. These techniques look at meeting transcripts to find words that appear often or are especially important in the context [23]. By highlighting these keywords, systems can make it easier for users to quickly grasp the main topics covered in meetings.

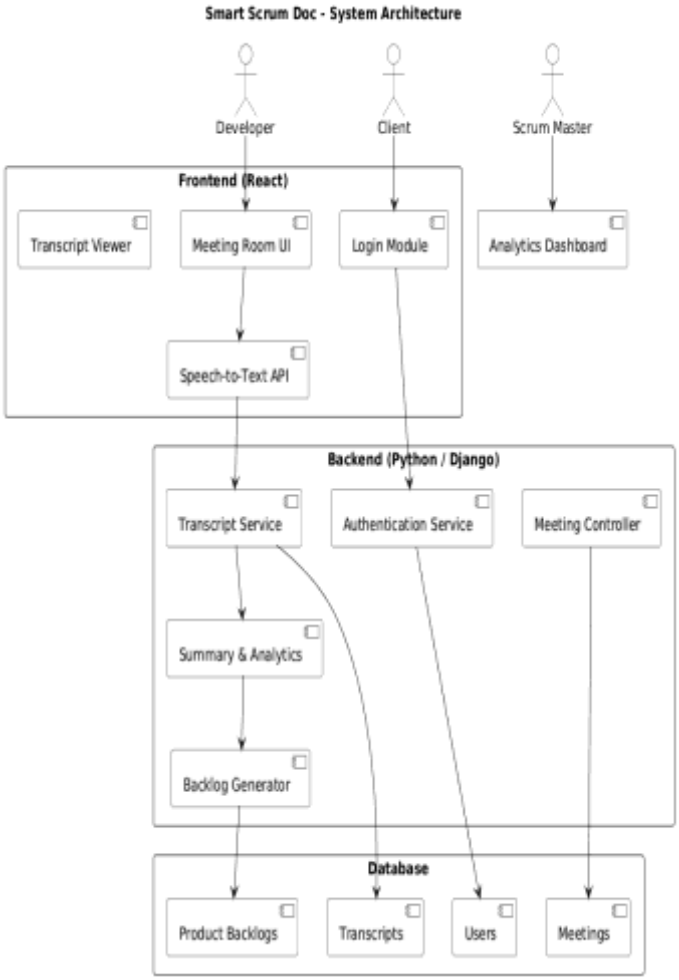
B. Proposed System

SmartScrumDoc is a system that uses Artificial Intelligence to automatically create documentation from Scrum meetings. Traditional ways of taking notes during meetings are done by hand, which can result in missing important details or incomplete records. SmartScrumDoc uses smart technologies like speech recognition and natural language processing to automatically record and organize meeting discussions into clear, structured documents [1]. The system is built to help agile development teams by making it easier to record and review meeting discussions. Scrum meetings usually involve many people talking about project tasks, progress, and problems. Keeping these discussions in an organized way helps teams keep accurate records and communicate better [2]. The first step of SmartScrumDoc is to capture the audio from Scrum meetings. It records what people are saying and saves that audio for processing later. Speech recognition technology, as explained in [3], turns spoken words into written text, which forms the starting point for creating meeting documents. Once the audio is recorded, the system uses Automatic Speech Recognition (ASR) to turn the spoken conversation into text. ASR looks at how people speak and the structure of language to create accurate transcripts. Research in [4] shows that today's ASR systems use deep learning to improve how well they can transcribe speech. After the transcript is created, the system applies Natural Language Processing (NLP) to understand the text. NLP helps the system find the meaning behind words and pick out important information from the transcript [5]. A key part of the system is its ability to spot important discussions during meetings. It looks at how sentences are structured and the context around them to find statements that include important decisions, updates, or plans for the project [6]. SmartScrumDoc also includes speaker identification to know who is talking during the meeting. Speaker diarization, as described in [7], uses voice characteristics to tell the difference between multiple speakers. Identifying who is speaking makes the transcripts clearer by linking each statement to the right person. This helps team members understand who came up with specific ideas or decisions during the meeting [8]. Another key feature of the system is its ability to create summaries of meetings. The summarization part looks at the transcript and picks out the most important sentences that capture the main points of the discussion [9]. SmartScrumDoc uses extractive summarization to select the most relevant sentences based on how much context they carry. These sentences are then combined to make a brief summary that shows the main conversation from the meeting [10]. In addition to creating summaries, the system also finds action items that come up during a meeting. Action items are tasks or responsibilities given to team members during the discussion [11]. The action item detection module looks through the meeting transcript to spot statements that show task assignments or things that need to be followed up on. Techniques for extracting information, as mentioned in [12], help the system recognize these kinds of statements. Another important part of the system is automatically making product backlog items.

In agile development, the product backlog has a list of tasks and requirements that help guide future work [13]. The system reviews the meeting discussions to find statements about feature requests, improvements, or bug fixes. These statements are turned into entries in the backlog, which help development teams plan their next tasks [14]. To make it easier to use, the system shows the generated information in different formats. Users can view the original meeting transcript, summary notes, listed action items, and backlog entries [15]. The system also has a user authentication module to make sure only authorized people can access the platform. Authentication methods protect the meeting data and control who can view it [16]. Once a user is authenticated, they can start a new meeting or join an existing one through the system's interface. The meeting interface lets participants talk and work together effectively [17]. During the meeting, the system automatically records the audio and prepares it for further analysis. This ensures that no important part of the conversation is left out [18]. After the meeting is over, the system processes the recorded audio to create a transcript and other related results. This automatic processing cuts down on the need for manual note-taking [19]. The system also includes a search feature that lets users find specific information in the meeting transcripts. Search algorithms described in [20] help users quickly find what they need. Another helpful feature is the ability to download meeting documents. Users can export meeting summaries and transcripts for later use [21]. The system's architecture is built using a modular design where each part does a specific job. Modular designs, as discussed in [22], make the system easier to scale and maintain. Using speech recognition and natural language processing technologies, the system changes unstructured conversations into organized documentation. This combination helps make meeting management more efficient [23]. The system also improves transparency in Scrum teams by clearly documenting discussions, decisions, and task assignments. Clear documentation helps team members keep track of how the project is progressing [24]. Overall, SmartScrumDoc offers a complete solution for automating the documentation of Scrum meetings. By mixing advanced AI technologies with easy-to-use interfaces, the system boosts productivity and ensures that important meeting details are properly recorded and used [25]. To make the system work better, the model uses advanced text processing methods. These help remove extra or repeated information from meeting transcripts. Steps like splitting text into words, removing common words that don't add much, and breaking sentences into parts help prepare the text for deeper language analysis [26]. The preprocessing stage makes the Natural Language Processing part more efficient. When text is cleaned and organized properly, algorithms can find meaningful patterns in the conversation more easily [27]. This leads to better accuracy when picking out the main points from the meeting transcript. A key part of the system is keeping structured records of past meetings. By storing transcripts, summaries, and backlog items in one place, users can look back on previous discussions and track the progress of projects over time [28]. The storage part of the system ensures all outputs are kept safely for future use. Secure ways of storing data help protect sensitive meeting details while letting the right people access historical records when needed [29]. The system also helps team members work together better by providing clear information on meeting results. When discussions are automatically turned into summaries and task lists, everyone can easily see what they need to do and follow up on their tasks [30]. Another benefit is that the system reduces communication issues within Scrum teams. By clearly writing down decisions, actions, and backlog items, everyone has the same information [31]. Including automatic documentation in Scrum meetings also helps with better project management. Development teams can use the generated backlog items to organize work more effectively and prioritize tasks based on project needs [32]. Moreover, the system supports ongoing improvement in agile environments. By looking at many meeting transcripts over time, the system can help teams spot repeated challenges or commonly discussed problems in their projects [33]. Another advantage is that it helps new team members understand past project discussions. Having access to summarized meeting records lets newcomers quickly learn about previous decisions and what the project needs [34]. Overall, the system shows how AI can be used to

IV. METHODOLOGY

A. *Architecture*



Module 1: Login Module

The Login Module is responsible for providing secure access to the SmartScrumDoc system. It allows different types of users such as clients, developers, and Scrum masters to log into the system using their credentials. This module verifies the identity of users before granting access to the application. It communicates with the backend authentication service to validate user information stored in the database. By ensuring proper authentication, this module prevents unauthorized access to sensitive meeting data. Once a user is successfully authenticated, the system allows them to access the meeting interface and other system features. The module also helps maintain session management during user interaction with the system. Security and privacy of meeting information are maintained through this process. This module acts as the gateway through which users enter the SmartScrumDoc platform.

Module 2: Meeting Room UI

The Meeting Room UI module is the interface where Scrum meetings happen. It lets users join or start meetings through the SmartScrumDoc system. During the meeting, people can talk and discuss project updates, tasks, and decisions. The module records the audio from the conversation. It connects with speech processing tools that help capture the discussion. This interface creates a simple and easy-to-use environment that feels like a real meeting room. The module also helps participants interact smoothly with each other. It serves as the main source for capturing meeting discussions for documentation. By gathering conversation data, the module starts the process of automatically documenting the meeting.

Module 3: Speech-to-Text API

The Speech-to-Text API module turns spoken words from a meeting into text. It uses speech recognition technology to analyze the audio being captured. This module listens to the audio coming from the meeting interface and detects the words that are being spoken. These words are then turned into a written record. This process helps reduce the need for people to write down notes by hand during meetings. The resulting text serves as the main input for additional text analysis. The module makes sure the conversation is accurately turned into text. This text allows the system to carry out natural language processing tasks. By changing speech into text, the module helps automatically record Scrum meetings.

Module 4: Transcript Viewer

The Transcript Viewer module allows users to view the textual record of the meeting discussion. After the speech-to-text process is completed, the generated transcript is displayed through this module. Users can read the full conversation that occurred during the meeting. This helps team members review important discussions and decisions. The module also allows users to quickly navigate through the transcript content. It improves transparency by providing a clear record of meeting interactions. The transcript can be used for reference in future project discussions. It also supports the summarization and backlog generation process. Overall, the module provides easy access to the meeting documentation generated by the system.

Module 5: Authentication Service

The Authentication Service is a part of the backend system that checks if users are who they say they are. It gets login requests from the frontend login module. The service compares the user's entered details with the information stored in the database. If the details match, the user is given access to the system. If not, access is blocked to keep the system secure. This service makes sure only people who are allowed can join meetings. It also keeps track of user sessions and who can do what. Having good authentication helps protect private information and keeps project discussions safe. This module is very important for keeping the whole system secure.

Module 6: Meeting Controller

The Meeting Controller module takes care of the whole meeting process in the system. It connects the front-end meeting interface with the back-end services. This module deals with tasks like creating, joining, and managing meetings. It makes sure meeting data is correctly handled and saved. The Meeting Controller also talks to other parts of the system, like the transcript and analytics modules. It works as a bridge between different parts of the system. By managing all meeting activities, it helps keep everything running smoothly. The module plays a key role in keeping control over meeting sessions. It is a vital part of the system for organizing all meeting-related tasks.

Module 7: Transcript Service

The Transcript Service module is in charge of managing the text that comes from converting speech to written words. It gets the text from the speech recognition module and works with it further. This service saves the transcript information in a database so it can be used later. It also shares the transcript data with other parts of the system, like the analytics and summarization services. The module makes sure the transcript is neatly arranged and formatted. It helps keep a full record of what was discussed in meetings. By saving transcripts, the system lets users look back at meeting talks whenever they need to. The module also helps find the data when it's needed. It's important for keeping track of meeting records.

Module 8: Summary & Analytics Service

The Summary and Analytics Service uses natural language processing to examine meeting transcripts. It finds important sentences, main topics, and decisions made during the meeting. This part of the system handles a lot of text and pulls out useful information. It creates a short summary of what was discussed. This summary allows team members to quickly see the key results of the meeting. The analytics part also looks for patterns and trends in the conversation. By turning long transcripts into clear summaries, the module boosts productivity. It makes sure important details aren't missed in long talks. This module is vital for turning raw meeting notes into valuable knowledge.

Module 9: Backlog Generator

The Backlog Generator module automatically finds tasks and requirements mentioned during meetings. It looks through the processed meeting notes to spot statements about future work or development tasks. These statements are turned into items in the product backlog. The generated backlog items help Scrum teams organize project needs in a better way. This module supports agile practices by connecting meeting discussions to actual tasks. It also cuts down on the work needed to create backlog entries by hand. The backlog items created can be saved and followed up on later. This helps teams keep their development process organized and structured. Overall, the module links meeting notes with project planning activities.

Module 10: Database

The Database module keeps all the essential data that the SmartScrumDoc system creates. It has organized tables like Users, Meetings, Transcripts, and Product Backlogs. The Users table holds login details and personal information for everyone who uses the system. The Meetings table keeps track of all the meetings that take place. The Transcripts table saves the written notes from meetings, which are made by the speech recognition tool. The Product Backlogs table includes the tasks and items found during meeting reviews. The database helps store and find data quickly. It also helps the system's background services by giving easy access to organized data. Managing the database well makes sure that the system's information is always reliable and stays consistent. This part of the system is where all data is stored long-term.

V. RESULTS

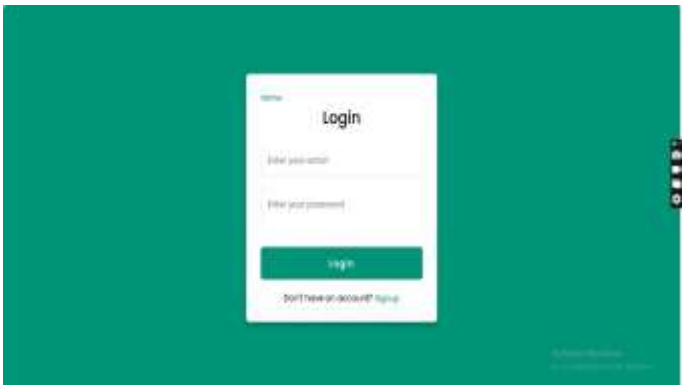


Fig-1 Login module

Fig-2 Dashboard

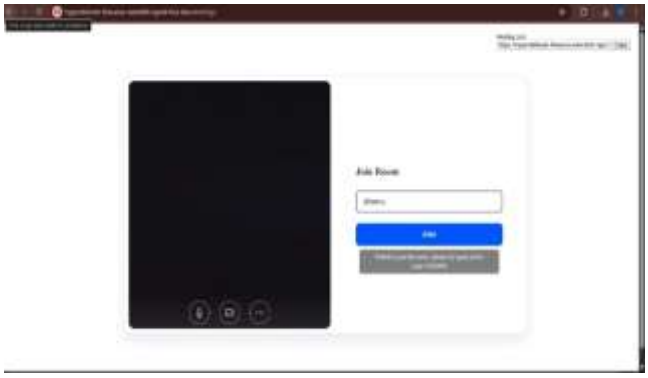


Fig-3 Join Room

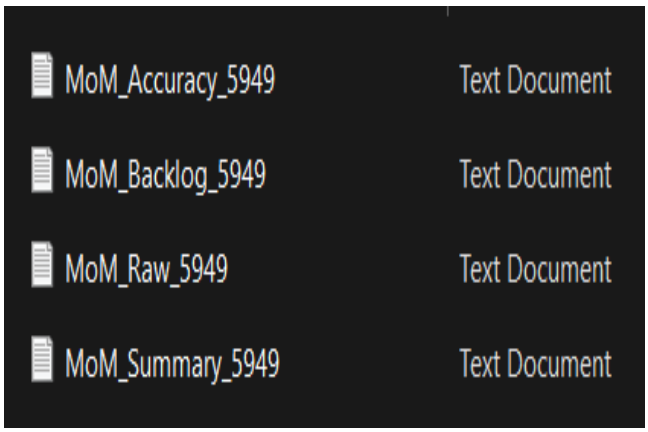


Fig-4 Mom files

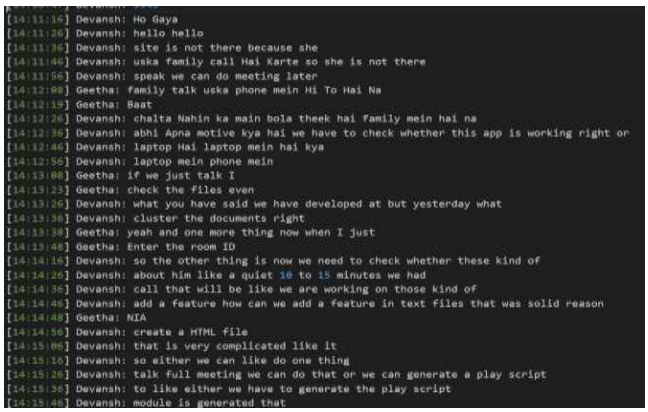


Fig-5 Raw Text

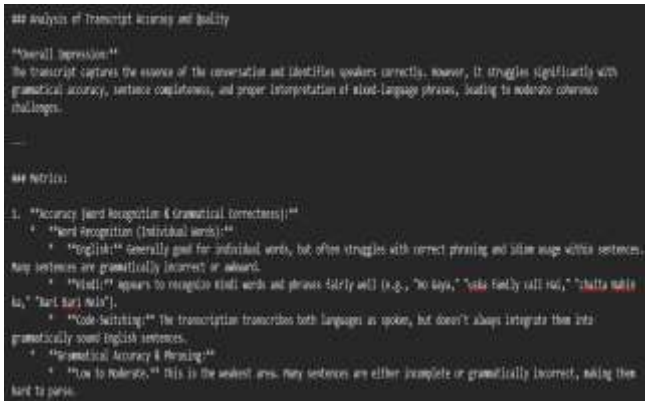


Fig-6 Accuracy file

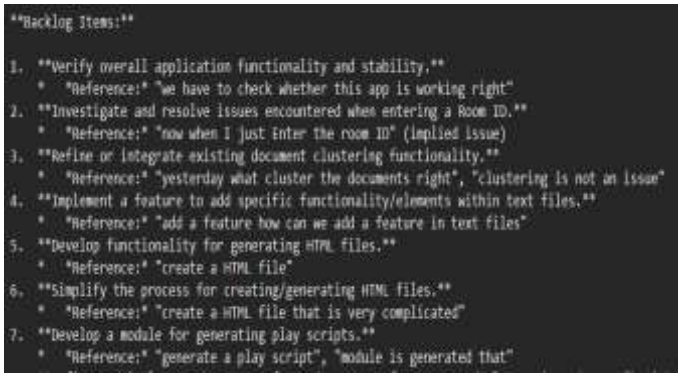


Fig-7 Product Backlog list

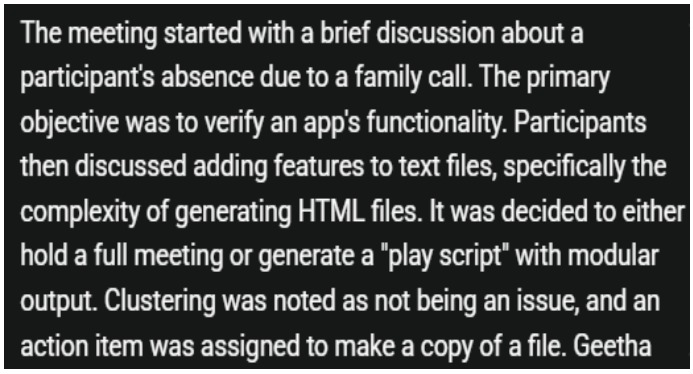


Fig-8 Summary

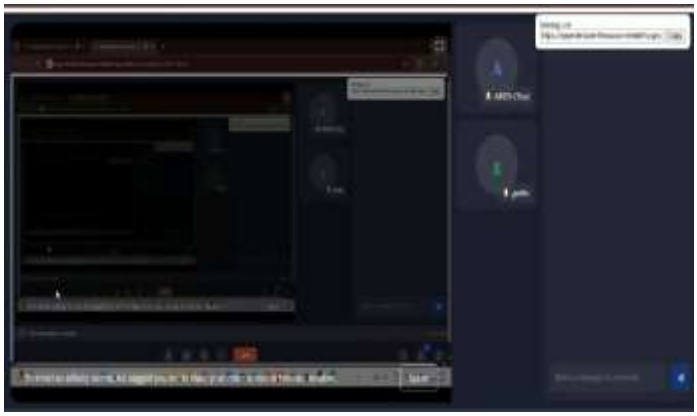


Fig-9 Meeting Room

VI. CONCULSION

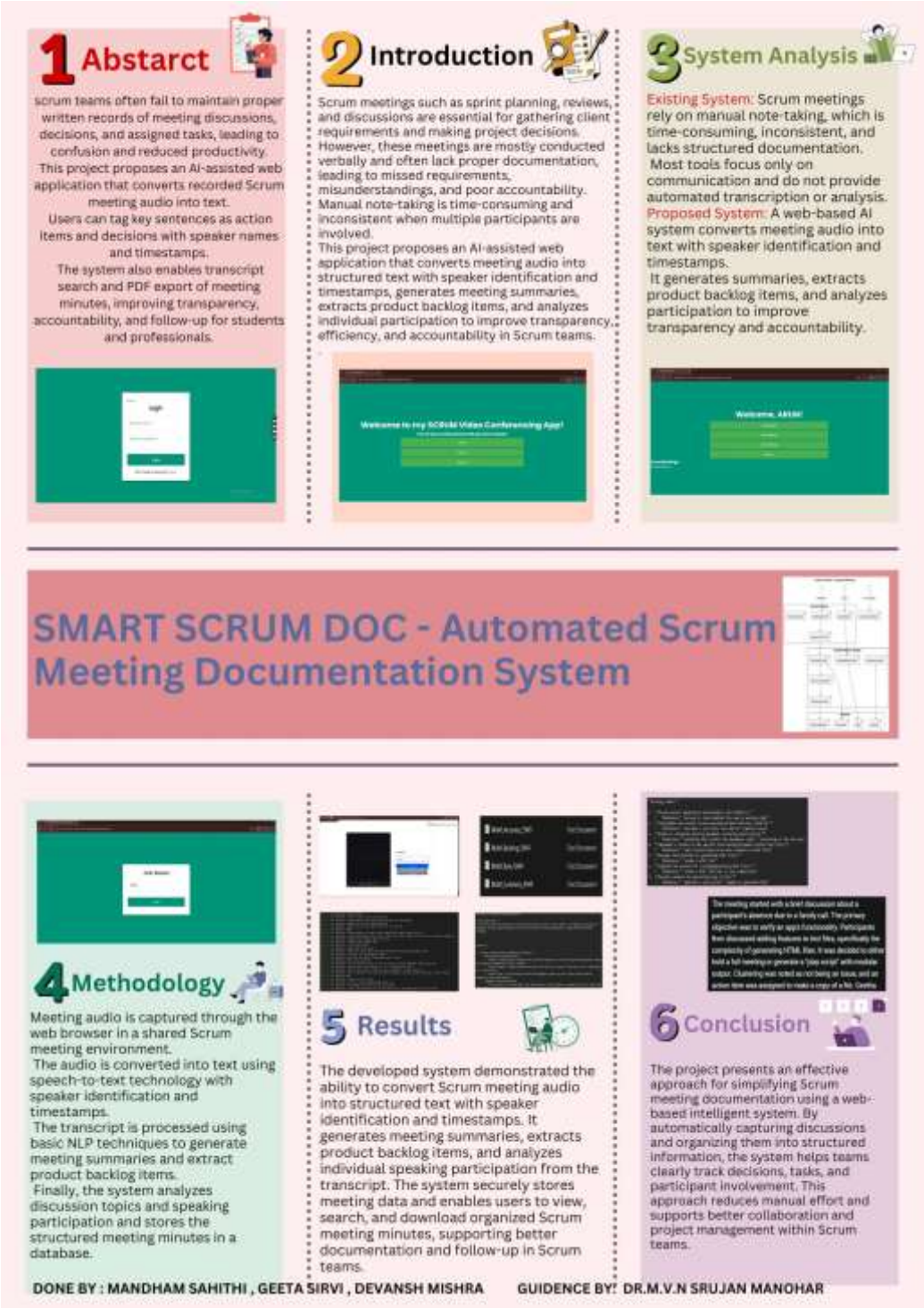
The SmartScrumDoc system was created to tackle the difficulties of documenting Scrum meetings in software development projects. Typically, keeping track of meeting notes by hand is time-consuming and can lead to mistakes or left-out details. By using automation in the documentation process, the system seeks to make things more efficient and accurate. It captures conversations from meetings and turns them into organized text. This helps reduce the workload on team members who would otherwise have to write down meeting minutes manually. The automatic documentation ensures that important points from the meetings are kept safe. It also makes the meeting records more reliable for Scrum teams.

The SmartScrumDoc system was created to tackle the difficulties of documenting Scrum meetings in software development projects. Typically, keeping track of meeting notes by hand is time-consuming and can lead to mistakes or left-out details. By using automation in the documentation process, the system seeks to make things more efficient and accurate. It captures conversations from meetings and turns them into organized text. This helps reduce the workload on team members who would otherwise have to write down meeting minutes manually. The automatic documentation ensures that important points from the meetings are kept safe. It also makes the meeting records more reliable for Scrum teams. A key part of the system is the use of speech recognition technology. This technology turns spoken words into written text with very little help from people. This allows the system to handle meeting discussions quickly and effectively. The written transcript then serves as the base for further work. By removing the need for manual transcription, the system saves time and effort. It also ensures that discussions are neatly formatted for easy use. The system also uses Natural Language Processing to make the most of the text it generates. These advanced techniques help the system find important points, action steps, and decisions from the meeting discussions. This ability to analyze large volumes of text speeds up the system's effectiveness. It also helps turn raw conversation data into structured and useful information. As a result, meeting results are easier to understand and manage. The SmartScrumDoc system also offers automated summarization of meetings. Instead of going through long transcripts, users can get quick summaries created by the system. These summaries highlight the most important parts of the meeting. This helps team members save time and focus on critical decisions. The summarization process boosts productivity within Scrum teams. It ensures that key information is clearly communicated. Another major benefit of the system is the automatic creation of product backlog items. Scrum meetings often include talks about tasks, features, and future improvements that need to be part of the project. Manually spotting these tasks can be slow. The SmartScrumDoc system automatically picks up on such discussions and turns them into backlog entries. This helps teams keep their list of development tasks organized. It also links meeting discussions to the overall project plans. The system’s architecture is designed to work smoothly between different parts like the user interface, the backend processing, and the data storage. Each part has a specific job in handling meeting data. The user interface lets people interact with the system in a simple and easy way. The backend handles speech recognition, text processing, and analysis. The database keeps transcripts, summaries, and backlog items for later use. This well-organized setup helps the system be scalable and easy to maintain. The system also helps team members work better together. By making meeting records and summaries available, all team members can stay updated on discussions, even if they couldn’t attend the meeting. This ensures nothing important gets missed. The system supports better communication within development teams. Better communication ultimately leads to improved project management. Another good idea is to connect SmartScrumDoc with other project management tools. When the system is linked to task management platforms, backlog items can be moved automatically into the development process. This helps make the connection between what's discussed in meetings and the actual work being done. This kind of integration would really help make agile project management better. Even though the system has some good features, there are certain issues that need to be fixed in the future. Adding real-time analytics can also give useful insights during meetings. These improvements will make the system smarter and more efficient. Keeping on improving AI models will help the system get even better over time.In short, the SmartScrumDoc system shows how AI can be used to make meeting notes easier in agile settings. By using speech recognition and natural language processing, the system turns meeting talks into written notes, summaries, and backlog items. This makes teamwork more efficient, clear, and collaborative. The system offers a real solution for keeping track of Scrum meetings.

VII. REFERENCES

1. D. Minkov, “Transforming meetings and docs with AI speech-to-text,” Medium,Sep.5,2023.
2. T. van Can and F. Dalpiaz, “Locating requirements in backlog items: Content analysis and experiments with large language models,” Information and Software Technology, vol. 179, p. 107644, Mar. 2025.
3. R. E. Stanly, “Building an AI-powered Microsoft Teams meeting summarizer (Transformers + NER + Azure Speech-to-Text),” Medium, Jan. 17, 2026.
4. A. Vaddi and K. Swapna, “Automatic Meeting Minutes Generation,” International Journal for Research in Applied Science & Engineering Technology (IJRASET), vol. 11, no. 4, pp. 1–8, 2023.
5. R. Khan and M. S. Khan, “AI-Powered Virtual Meeting Summarization System,” SSRN, Jul. 2025.
6. A. Pandya and N. Gawande, “Automatic Generation of Minutes of Meetings,” Int. J. Sci. Res. Sci. Eng. Technol., vol. 9, no. 2, pp. 93–99,Mar.–Apr.2022.
7. H. B. Chandaka, D. R. Kandimalla, V. Turaka, and N. Shivani, “Voice Meeting Summarization,” Int. J. Curr. Sci. (IJCSPUB), vol. 13, no. 2, pp. 616–623, May 2023.
8. S. Rezazadegan, S. Berkovsky, J. C. Quiroz, A. B. Kocaballi, Y. Wang, L. Laranjo, and E. Coiera, “Automatic Speech Summarisation: A Scoping Review,” arXiv:2008.11897, Aug. 2020.
9. S. Furui, T. Kikuchi, Y. Shinnaka, and C. Hori, “Speech-to-text and speech-to-speech summarization of spontaneous speech,” IEEE Trans. Speech Audio Process., vol. 12, no. 4, pp. 401–408, Jul. 2004.
10. K. Shinde, L. Besacier, O. Bojar, T. Thonet, and T. Ghosal, “Findings of the Third Automatic Minuting (AutoMin) Challenge,” arXiv:2509.13814, Sep. 2025.
11. J. Carletta, S. Ashby, S. Bourban, M. Guillemot, M. Kronenthal, G. Lathoud, and M. Lincoln, “The AMI Meeting Corpus: A Pre-announcement,” Machine Learning for Multimodal Interaction, pp. 28–39, 2005.
12. G. Murray, S. Renals, J. Carletta, and J. Moore, “Incorporating speaker and discourse features into speech summarization,” Proceedings of the Human Language Technology Conference, pp. 367–374, 2005.
13. L. Laban, S. Berkovsky, and E. Coiera, “Meeting summarization: A review of approaches and challenges,” ACM Computing Surveys, vol. 55, no. 3, pp. 1–38, 2023.
14. H. Christensen, Y. Gotoh, and S. Renals, “Automatic summarization of meeting speech,” IEEE Workshop on Automatic Speech Recognition and Understanding, pp. 133–138, 2004.
15. M. Janin, D. Baron, J. Edwards, and D. Ellis, “The ICSI Meeting Corpus,” IEEE International Conference on Acoustics, Speech, and Signal Processing, pp. 364–367, 2003.
16. A. Gupta and R. Kumar, “AI-based meeting transcription and summarization system,” International Journal of Computer Applications, vol. 184, no. 22, pp. 15–20, 2022.
17. P. Liu and W. Chen, “Automatic meeting summarization using natural language processing techniques,” IEEE International Conference on Artificial Intelligence and Knowledge Engineering, pp. 45–50, 2021.
18. S. Mehta and P. Shah, “Speech recognition and summarization system for business meetings,” International Journal of Advanced Computer Science and Applications, vol. 12, no. 7, pp. 325–332, 2021.
19. R. Patel and K. Sharma, “Automated extraction of action items from meeting transcripts using NLP,” Journal of Information Processing Systems, vol. 19, no. 1, pp. 44–58, 2023.
20. A. R. Pathak and S. Kulkarni, “AI-assisted documentation system for collaborative meetings,” Proceedings of the International Conference on Intelligent Systems and Applications, pp. 112–118, 2024.

POSTER


1 Abstarct
scrum teams often fail to maintain proper written records of meeting discussions, decisions, and assigned tasks, leading to confusion and reduced productivity.
This project proposes an AI-assisted web application that converts recorded Scrum meeting audio into text.
Users can tag key sentences as action items and decisions with speaker names and timestamps.
The system also enables transcript search and PDF export of meeting minutes, improving transparency, accountability, and follow-up for students and professionals.
2 Introduction
Scrum meetings such as sprint planning, reviews, and discussions are essential for gathering client requirements and making project decisions. However, these meetings are mostly conducted verbally and often lack proper documentation, leading to missed requirements, misunderstandings, and poor accountability. Manual note-taking is time-consuming and inconsistent when multiple participants are involved.
This project proposes an AI-assisted web application that converts meeting audio into structured text with speaker identification and timestamps, generates meeting summaries, extracts product backlog items, and analyzes individual participation to improve transparency, efficiency, and accountability in Scrum teams.
3 System Analysis
Existing System: Scrum meetings rely on manual note-taking, which is time-consuming, inconsistent, and lacks structured documentation.
Most tools focus only on communication and do not provide automated transcription or analysis.
Proposed System: A web-based AI system converts meeting audio into text with speaker identification and timestamps.
It generates summaries, extracts product backlog items, and analyzes participation to improve transparency and accountability.
SMART SCRUM DOC - Automated Scrum Meeting Documentation System
4 Methodology
Meeting audio is captured through the web browser in a shared Scrum meeting environment.
The audio is converted into text using speech-to-text technology with speaker identification and timestamps.
The transcript is processed using basic NLP techniques to generate meeting summaries and extract product backlog items.
Finally, the system analyzes discussion topics and speaking participation and stores the structured meeting minutes in a database.
5 Results
The developed system demonstrated the ability to convert Scrum meeting audio into structured text with speaker identification and timestamps. It generates meeting summaries, extracts product backlog items, and analyzes individual speaking participation from the transcript. The system securely stores meeting data and enables users to view, search, and download organized Scrum meeting minutes, supporting better documentation and follow-up in Scrum teams.
6 Conclusion
The project presents an effective approach for simplifying Scrum meeting documentation using a web-based intelligent system. By automatically capturing discussions and organizing them into structured information, the system helps teams clearly track decisions, tasks, and participant involvement. This approach reduces manual effort and supports better collaboration and project management within Scrum teams.
DONE BY : MANDHAM SAHITHI , GEETA SIRVI , DEVANSH MISHRA
GUIDENCE BY: DR.M.V.N SRUJAN MANOHAR